Search Trees

Jianguo Lu

November 4, 2025

Overview

Binary Search Tree

Balanced Search Tree

AVL Tree

(2,4) trees

Red-black trees

Sorted/Ordered Map

Key operations:

- get(k): Returns the value v associated with key *k*
- put(k, v): Associates value v with key k, replacing and returning any existing value if the map already contains an entry with key equal to k.
- remove(k): Removes the entry with key equal to k, if one exists, and returns its value

Why not HashMap

- Drawbacks of Hash map:
  - Can't do vicinity search (e.g., lowerEntry(key))
  - Not efficient space-wise–need to allocate more memory than needed to reduce collision
- Now suppose that keys are from a total order.
- We can utilize this property to have an efficient data structure

Total order

If X is totally ordered under $\leq$, then the following statements hold for all a, b and c in X:

- If $a \leq b$ and $b \leq a$ then $a = b$ (antisymmetry);
- If $a \leq b$ and $b \leq c$ then $a \leq c$ (transitivity);
- $a \leq b$ or $b \leq a$ (totality).

Vicinity search example

```
import java.util.*;

public class VicinitySearch {
    public static void main(String[] args) {
        TreeMap<Integer, String> map = new TreeMap<>();
        map.put(10, "A");
        map.put(20, "B");
        map.put(30, "C");
        map.put(40, "D");

        int x = 25;

        System.out.println("floorKey:   " + map.floorKey(x));   //20
        System.out.println("ceilingKey: " + map.ceilingKey(x)); //30
        System.out.println("lowerKey:   " + map.lowerKey(x));   //20
        System.out.println("higherKey:  " + map.higherKey(x));  //30
    }
}
```

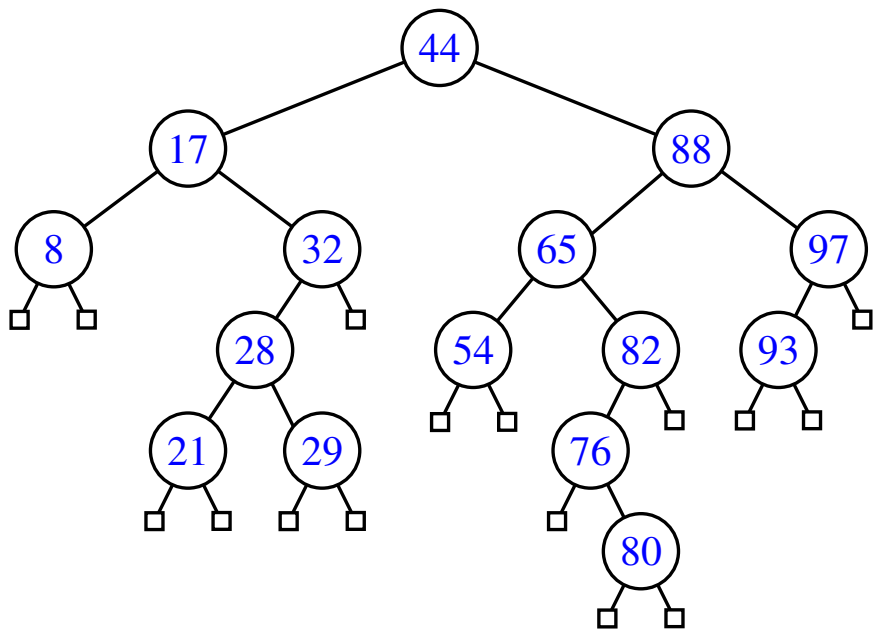
```
floorKey:   20
ceilingKey: 30
lowerKey:   20
higherKey:  30
```

Binary search tree

- Let u, v, and w be three nodes such that u is in the left subtree of v and w is in the right subtree of v. We have

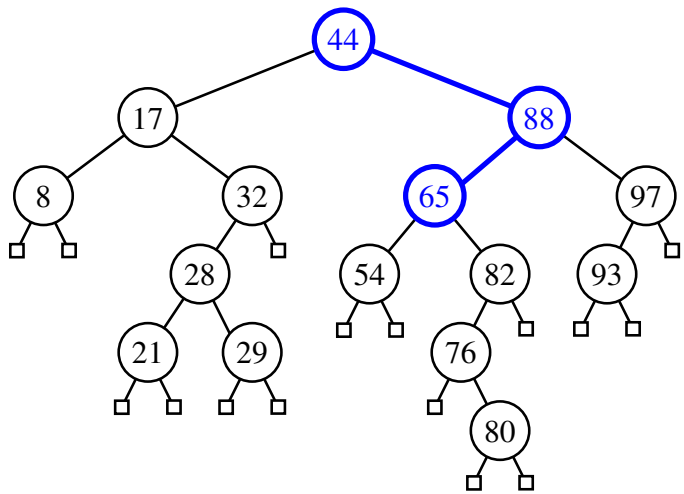
$$key(u) \leq key(v) \leq key(w) \tag{1}$$

- External nodes do not store items
- An in-order traversal visits the keys in increasing order

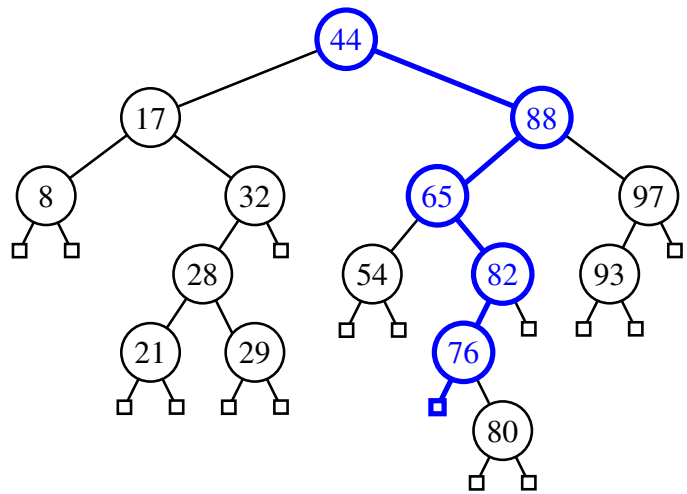


Tree Searching

- To search for a key k, we trace a downward path starting at the root
- The next node visited depends on the comparison of k with the key of the current node
- If we reach a leaf, the key is not found



Search for 65



Search for 68

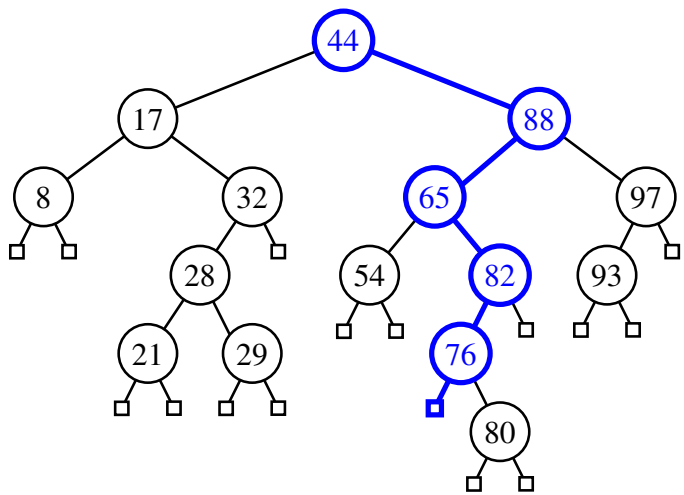
Binary tree search algorithm

```
Position<Entry<K,V>> treeSearch(Position<Entry<K,V>> p, K key) {
  if (isExternal(p))    return p;
  int comp = compare(key, p.getElement());
  if (comp == 0)    return p;
  else if (comp < 0)
              return treeSearch(left(p), key);
        else  return treeSearch(right(p), key);
}
```

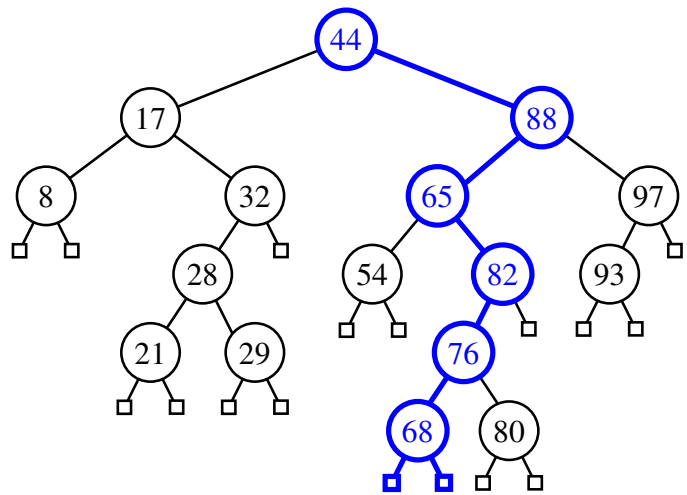
Insertion

```
TreeInsert(k, v)
        p = TreeSearch(root(), k)
        if k == key(p)
                Change value of p to (v)
        else
                expandExternal(p, (k,v))
```

- Search then expand
- Example: Insert 68:



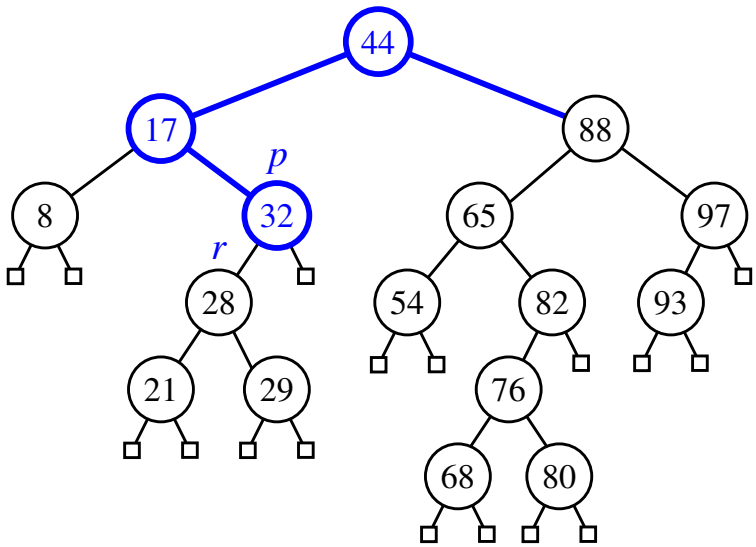
Search for 68



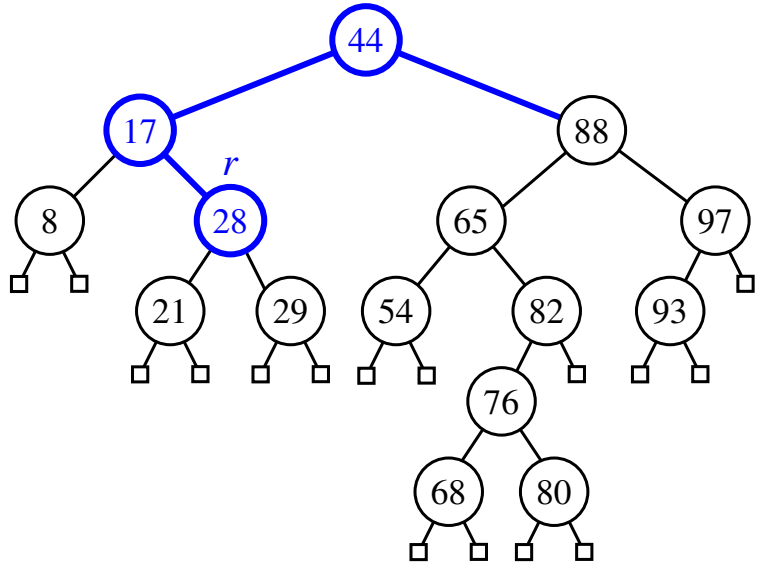
Expand the leaf

Deletion

- When there is an external child: move the subtree up



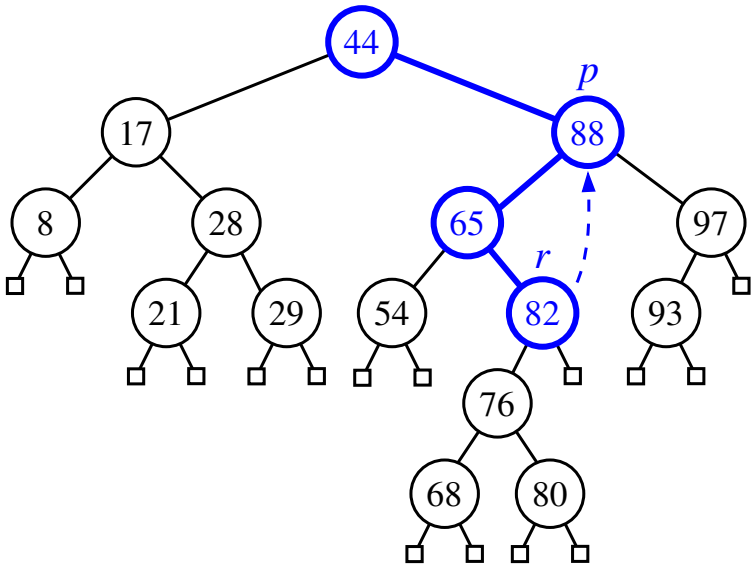
Before the deleting node 32



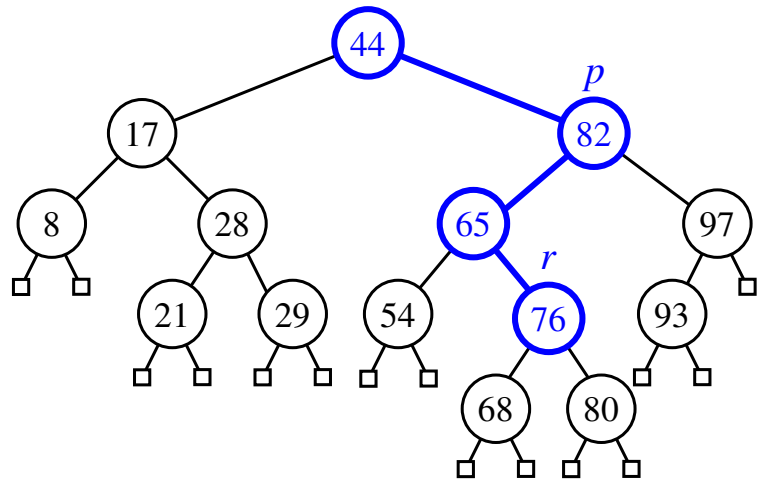
After the deletion

Deletion: when there is no external child

- Locate the predecessor (the right most child in the left branch)
- Replace the deleted position with the predecessor
- Delete the predecessor

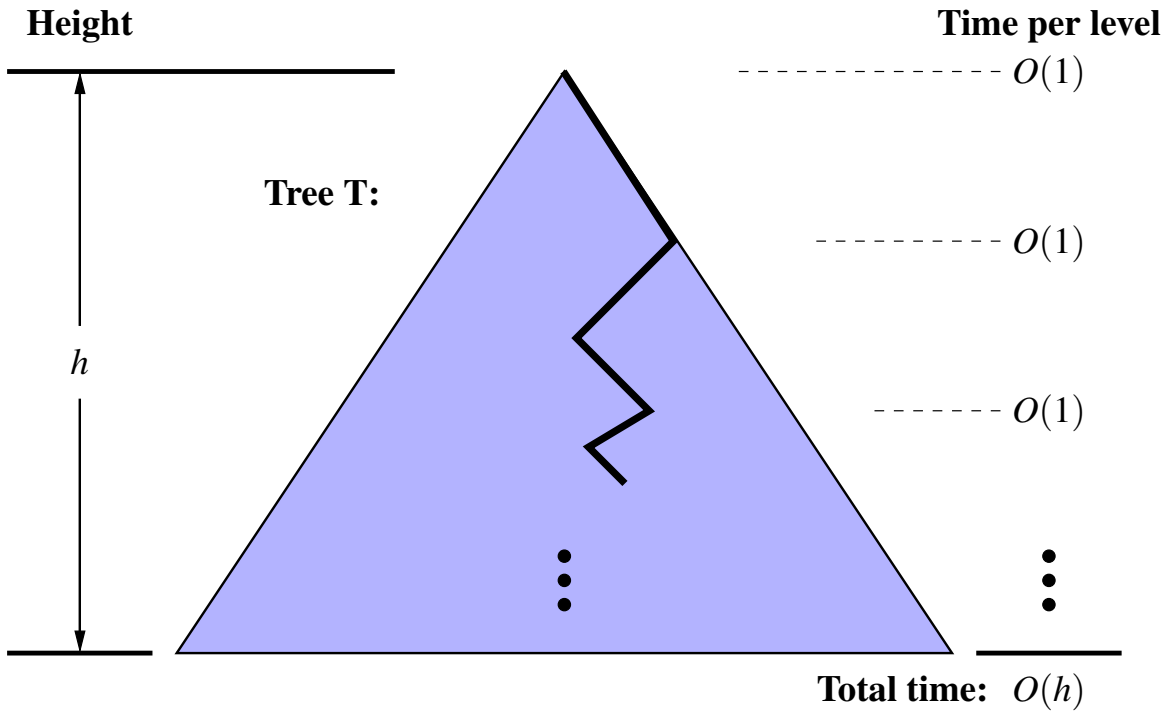


Before deleting 88



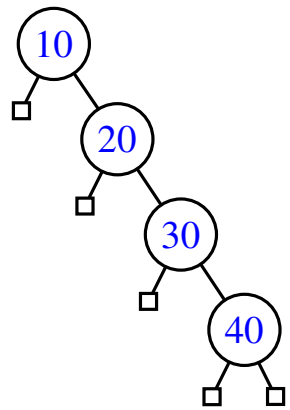
After deletion

Analysis of binary tree search



Complexities of BST

Method	Running Time
get, put, remove	O(h)
firstEntry, lastEntry	O(h)
ceilingEntry, floorEntry, lowerEntry, higherEntry	O(h)



- The complexity depends on the height of the tree
- The height could be big
- Worst case: $h = n$.
- Best case: $h = log(n + 1) - 1$. Recall that $n = 2^0 + 2^1 + \cdots + 2^h = 2^{h+1} - 1$.

Binary Search Tree

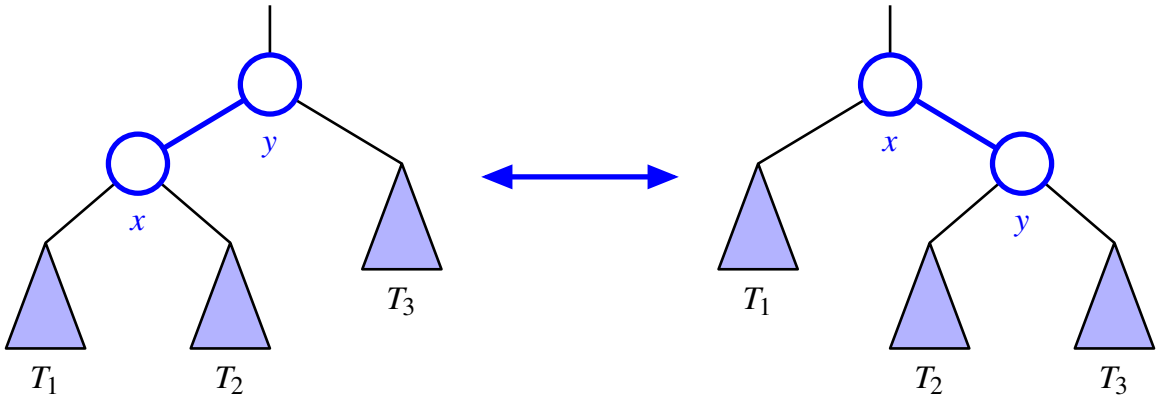
Balanced Search Tree

AVL Tree

(2,4) trees

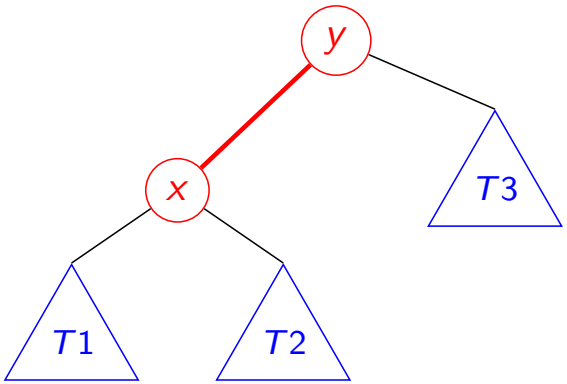
Red-black trees

Rotation



- from left to right: y becomes the right child of x after the rotation
- from right to left: x becomes the child of y
- Relink the subtree of items (T_2 in the example)
- Complexity of rotation operation: O(1)
  - a single rotation modifies a constant number of parent-child relationships

Detailed steps: Raise x above y



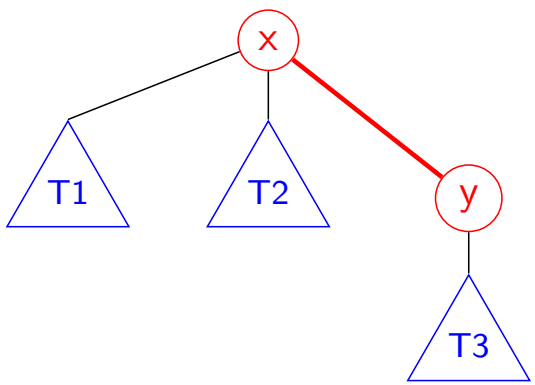
- When the x branch is high, we want to reduce the height, so that the tree is more balanced.
- We can achieve this by raising the x above y.

Raise x above y (step 1)

- Make x parent of y
- The code is `relink(x,y, righChild)` .

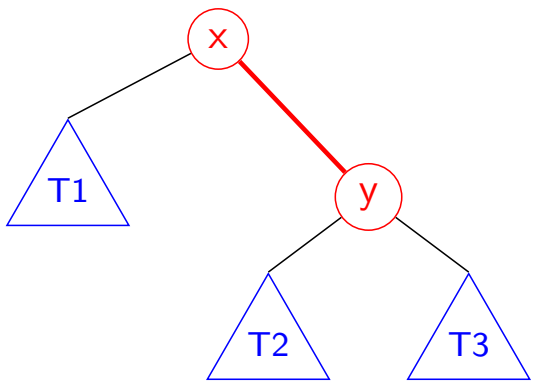
```
void relink(Node parent, Node child, boolean makeLeftChild) {
  child.setParent(parent);
  if (makeLeftChild) parent.setLeft(child);
  else parent.setRight(child);
 }
```

- The third argument is needed to link y as either left or right child.
  - In this case, it is to link y as the right child. (because it is the left rotation)
- Now T2 is not in the right place



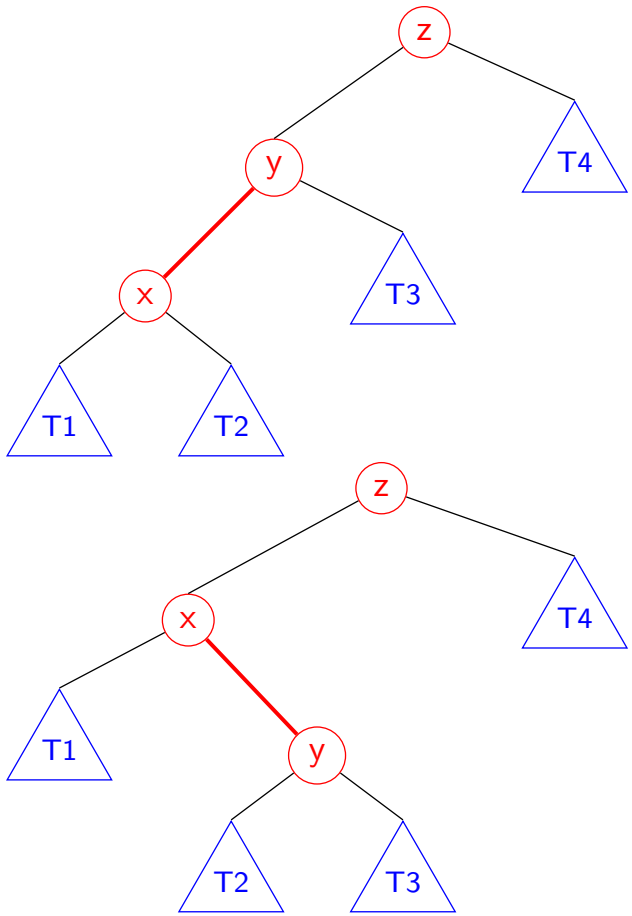
Raise x above y (step 2)

- Make T2 the left child of y.
- The code is `relink(y, x's right child, isLeft)`
- T2 is x's right child
- Make T2 the left child of y



Who is the parent of x? The complete picture

```
Node<Entry<K,V>> x = validate(p);
Node<Entry<K,V>> y = x.getParent();
Node<Entry<K,V>> z = y.getParent();

relink(z, x, y == z.getLeft());
```



Rotate operation

Rotate p above its parent (page 478 of the book):

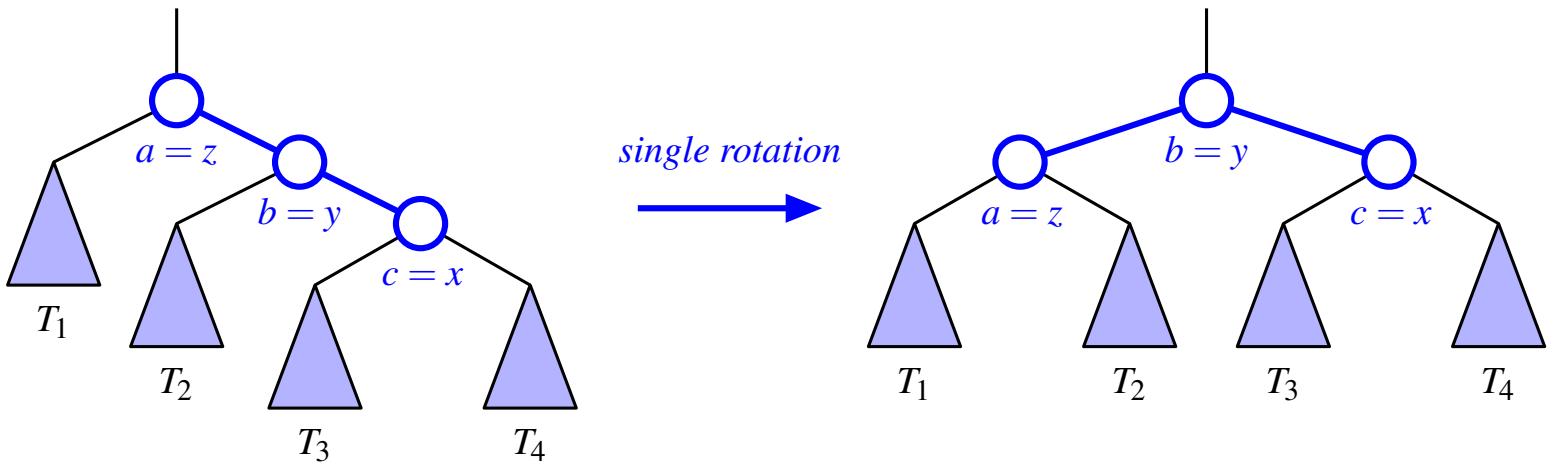
```
public void rotate(Position<Entry<K,V>> p) {
  Node<Entry<K,V>> x = validate(p);
  Node<Entry<K,V>> y = x.getParent();
  Node<Entry<K,V>> z = y.getParent();

  relink(z, x, y == z.getLeft());  // x becomes direct child of z

  // now rotate x and y, including transfer of middle subtree
  if (x == y.getLeft()) {
     relink(y,x.getRight(),true);//x's right child becomes y's left
     relink(x,y,false); //y becomes x's right child
  } else {
     relink(y,x.getLeft(),false); //x's left child becomes y's right
     relink(x,y,true);    // y becomes left child of x
 }
```

Recall that relink is to link a parent node with its child node with orientation either left or right.

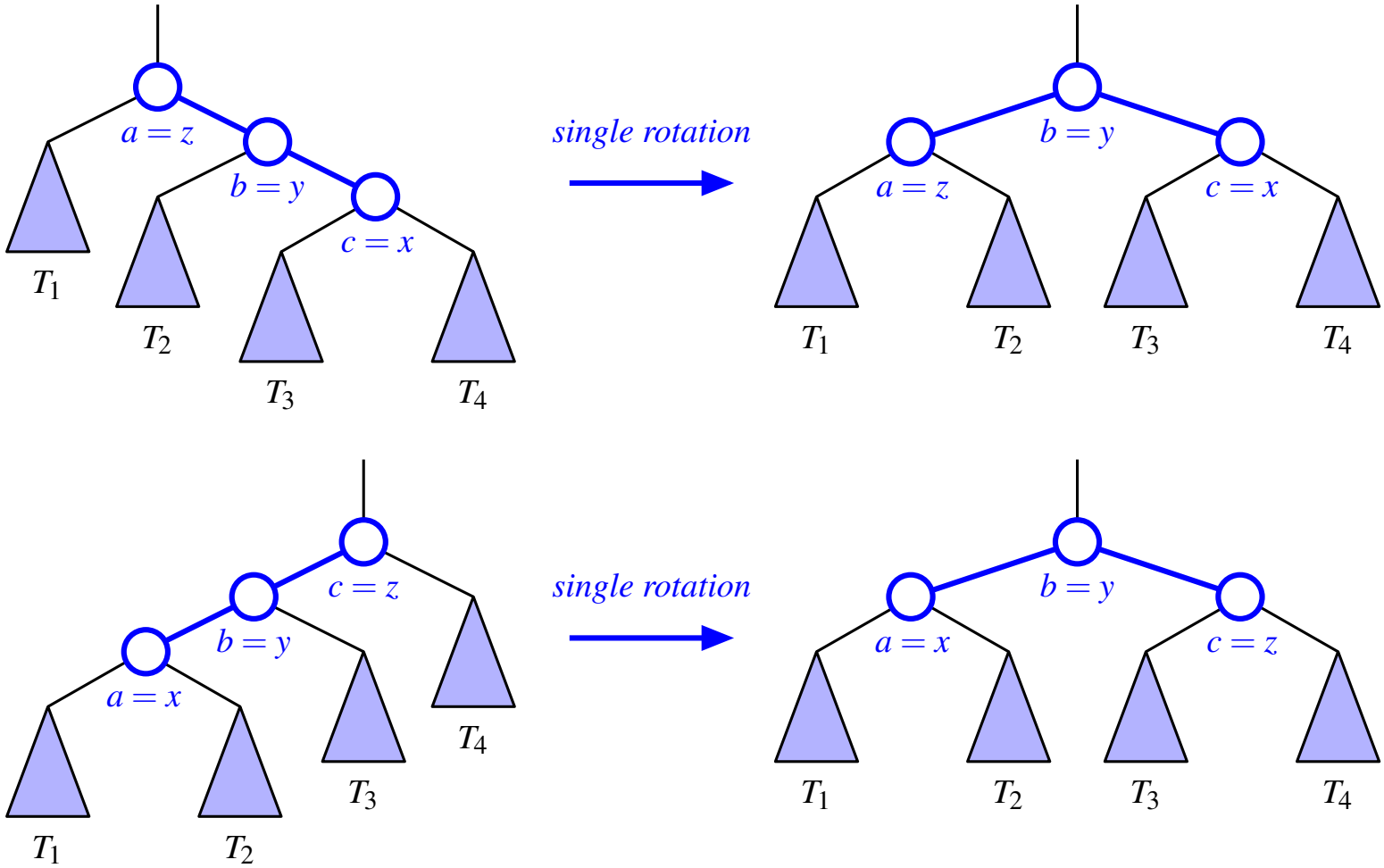
```
void relink(Node  parent, Node  child, boolean makeLeftChild)
  child.setParent(parent);
  if (makeLeftChild)  parent.setLeft(child);
  else  parent.setRight(child);
```

Tri-node restructuring

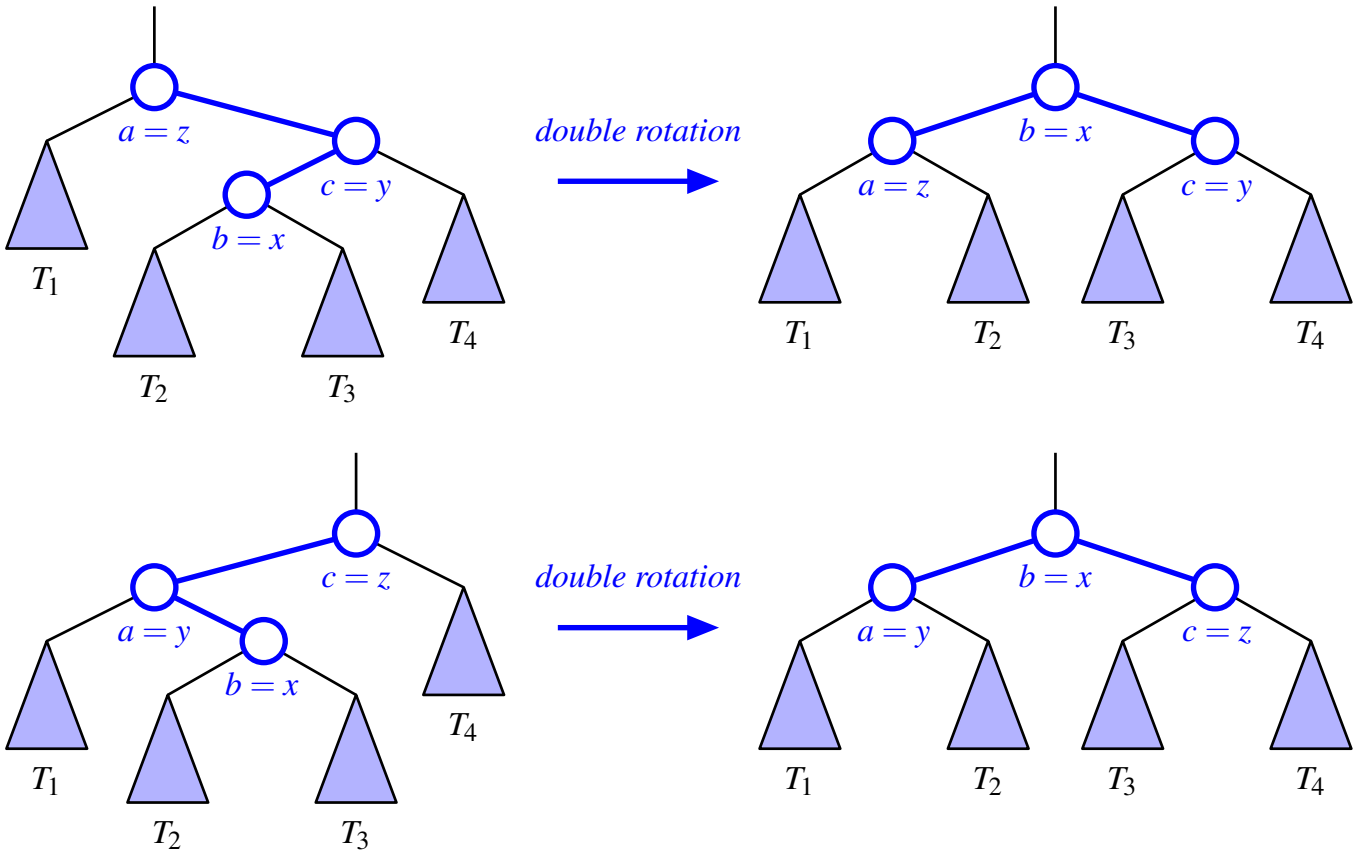
Let (a,b,c) be the inorder listing of x, y, z. Perform the rotations needed to make b the topmost node of the three



- Multiple rotations can be combined to provide broader rebalancing within a tree.
- One such compound operation we consider is a trinode restructuring.
- There are four possible orientations mapping x, y, and z to a, b, and c.


a = z
b = y
c = x
T1
T2
T3
T4
single rotation
b = y
a = z
c = x
T1
T2
T3
T4
c = z
b = y
a = x
T4
T3
T1
T2
single rotation
b = y
a = x
c = z
T1
T2
T3
T4

Double rotation



Binary Search Tree

Balanced Search Tree

AVL Tree

(2,4) trees

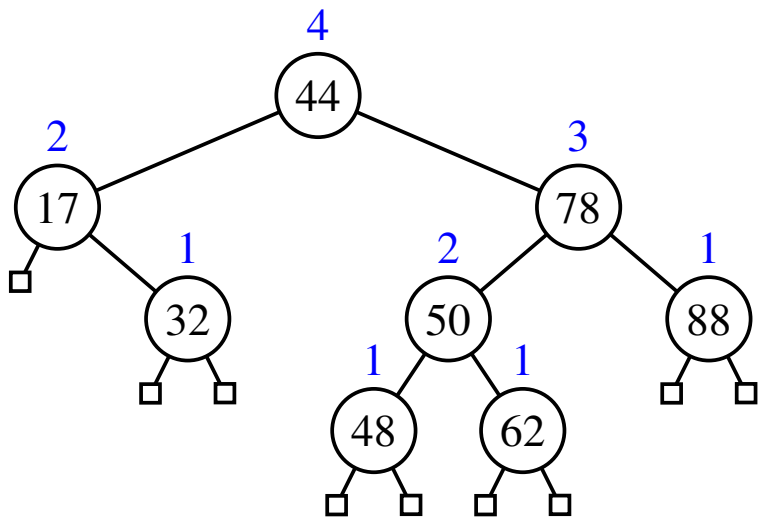
Red-black trees

AVL Tree

Definition: AVL Tree

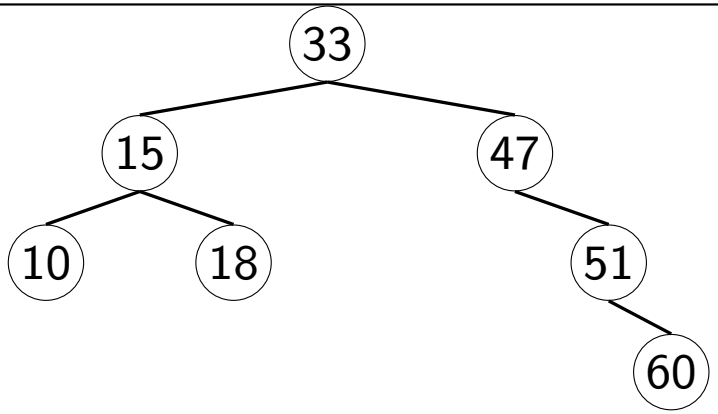
- AVL trees are height balanced binary search tree
- Height balanced: for every node v of T, the heights of the children of v can differ by at most 1

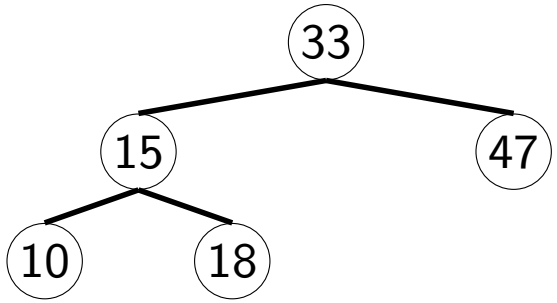
$$|height(left) - height(right)| \leq 1$$

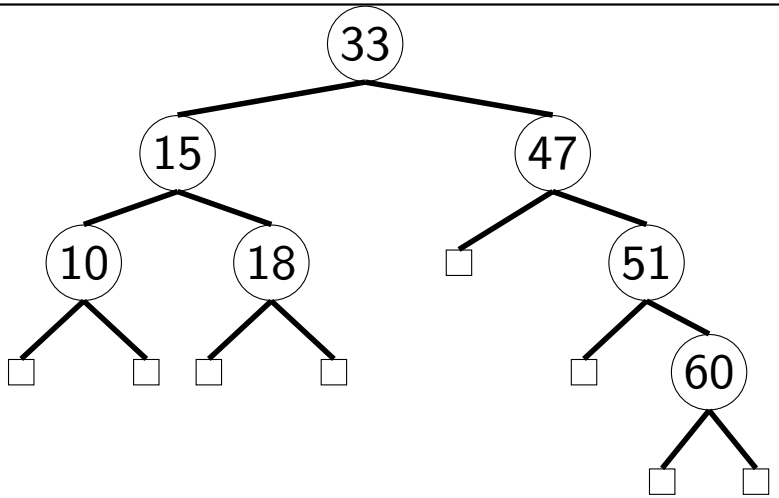


- Note that the external nodes have height zero.
- AVL named after the inventors Adelson-Velskii and Landis
- E.x. Give an BST that is not AVL.

AVL or not AVL?

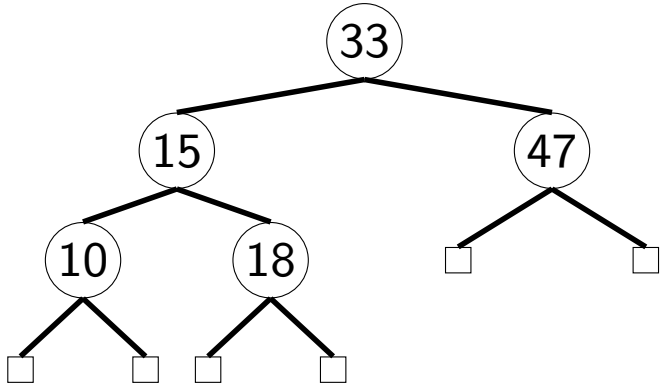






Not AVL

easier to reason when there are sentinels



AVL

count height, not number of descendants

The height of an AVL tree is $O(\log n)$

Let $n(h)$ denote the *minimum* number of internal nodes of an AVL tree of height h.

- $n(1) = 1$ and $n(2) = 2$
- For $n > 2$, an AVL tree of height h contains
  - the root node,
  - one AVL subtree of height $h - 1$
  - another of height $h - 2$ or $h - 1$
- Therefore

$$\begin{aligned} n(h) &= 1 + n(h-1) + n(h-2) \\ &> n(h-1) + n(h-2) \end{aligned} \tag{2}$$

The height of AVL tree

- Knowing $n(h-1) > n(h-2)$, we get

$$\begin{aligned} n(h) &> 2n(h-2) \\ &> 2^2 n(h-2\times 2), \\ &> 2^3 n(h-2\times 3) \\ &\dots \\ &> 2^i n(h-2i) \end{aligned} \tag{3}$$

When i=(h-1)/2, using the base case $n(1) = 1$ we get:

$$n(h) > 2^{(h-1)/2} n(1) = 2^{(h-1)/2} \times 1 = 2^{(h-1)/2} \tag{4}$$

Taking logarithms:

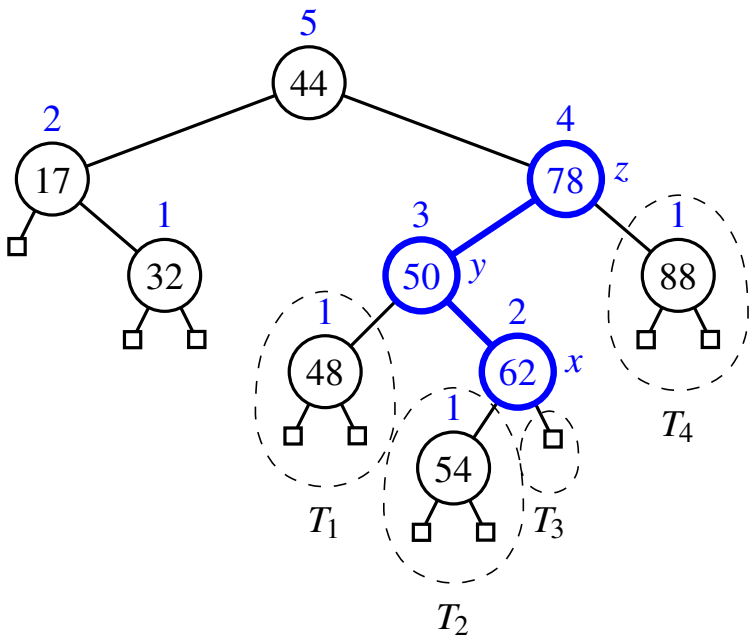
$$h < 2\log n + 1 \tag{5}$$

Thus the height of an AVL tree is $O(\log n)$

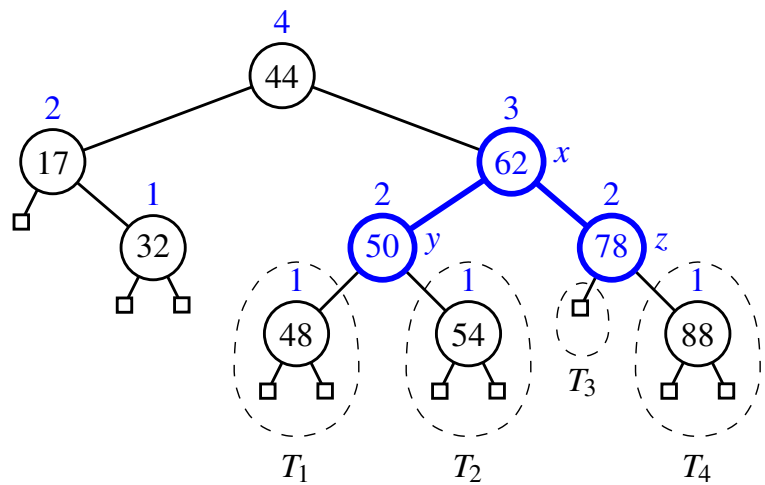
Insertion and rebalance

- Find z,y,x:
  - z: first unbalanced node encountered while travelling up the tree from w.
  - y: child of z with larger height,
  - x: child of y with larger height

Insert node 54

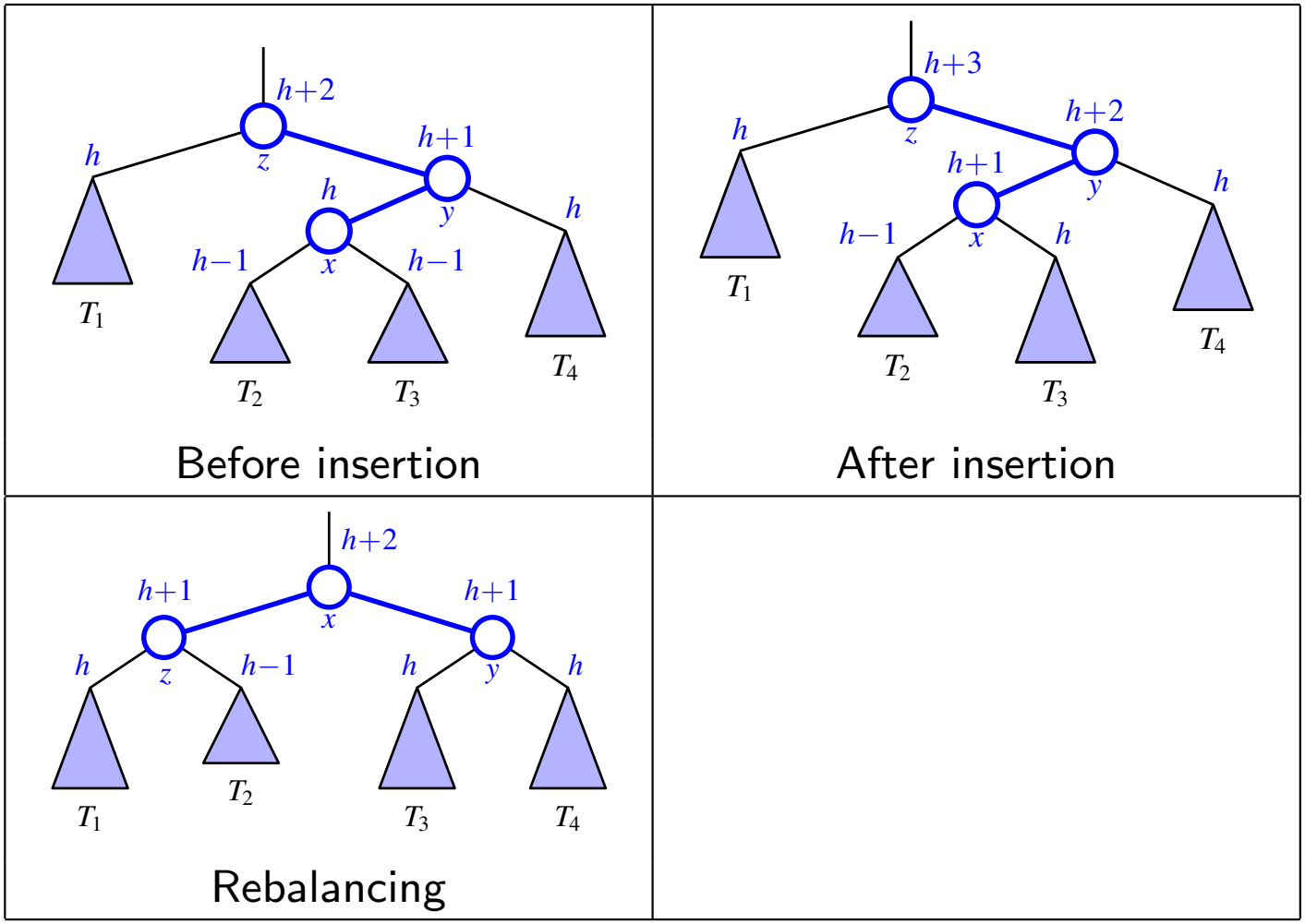


insert 54



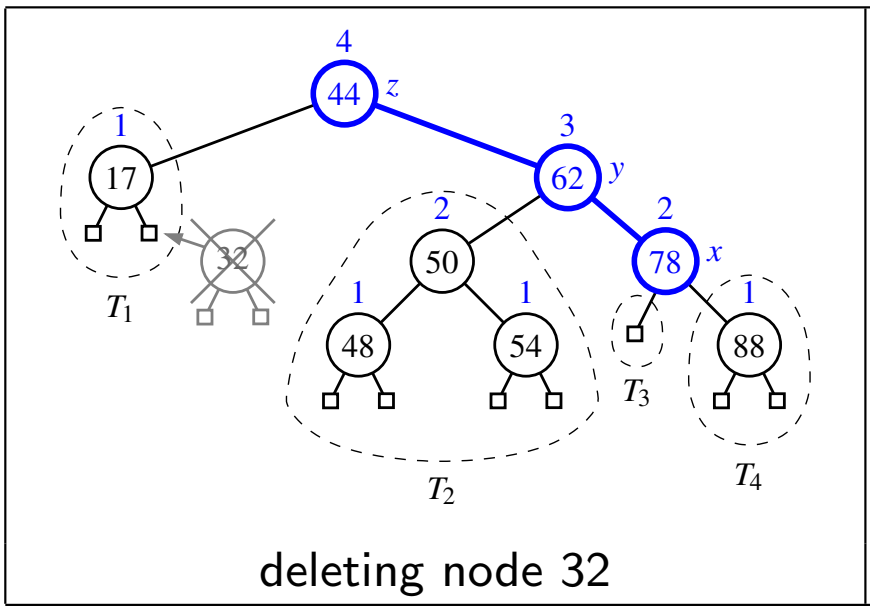
restructure

Rebalancing in general

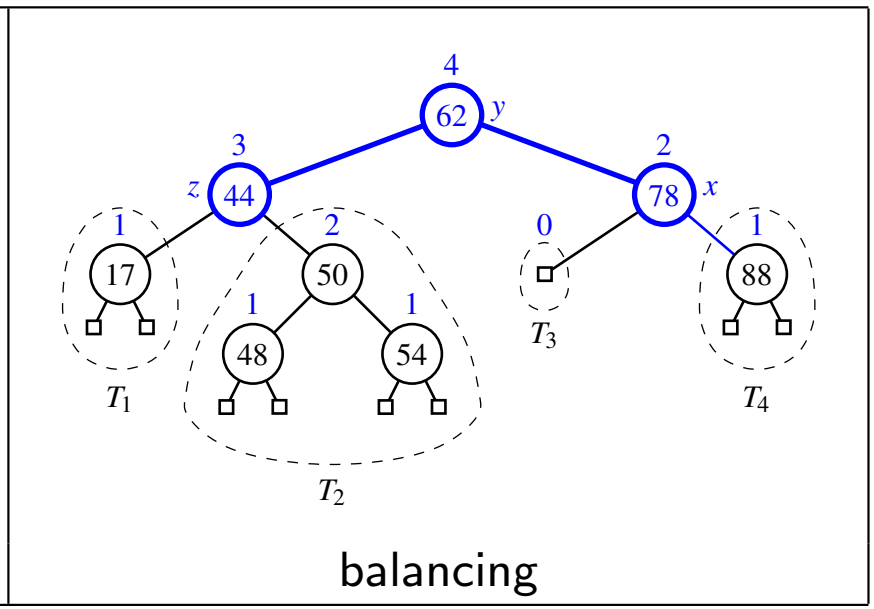


Deletion

- Find z,y,x:
  - z: first unbalanced node encountered while travelling up the tree from w.
  - y: child of z with larger height,
  - x: child of y with larger height
- Perform a trinode restructuring to restore balance at z
- As this restructuring may upset the balance of another node higher in the tree, we must continue checking for balance until the root of T is reached



deleting node 32

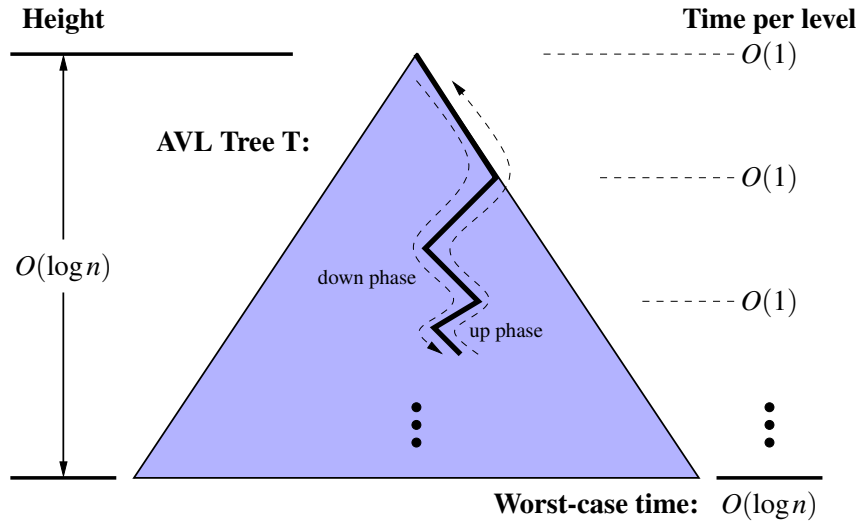


balancing

Performance of AVL trees

For an AVL tree storing n items

- Space: O(n)
- A single restructuring: O(1)
- height: O(log n)
  - Searching: O(log n)
  - Insertion: O(log n)
  - Remove: O(log n)
  - restructuring up the tree, maintaining heights is O(log n)



Ex. Insert 44, 17, 78, 32,50, 88, 48, 62, 54

Ex. Delete 32, 62

Binary Search Tree

Balanced Search Tree

AVL Tree

(2,4) trees

Red-black trees

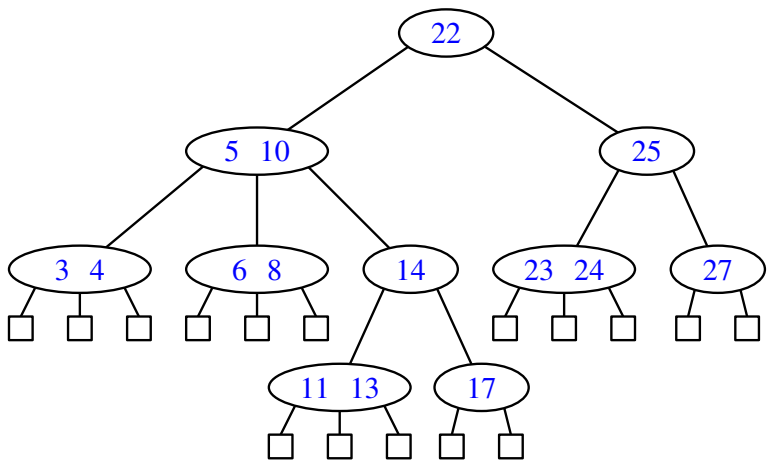
Multiway search tree

- Each internal node of T has at least two children.
- Each internal is a d-node. d is an integer such as 2, 3, 4.
- A d-node has d children $c_1, \ldots, c_d$, and stores an ordered set of $d-1$ keys

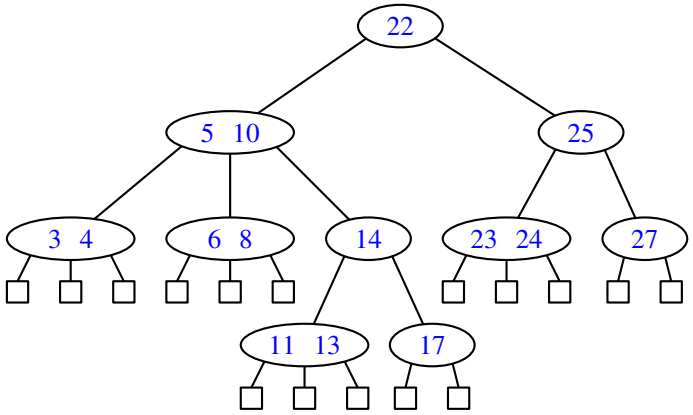
$$k_1 \leq k_2 \leq \cdots \leq k_{d-1}$$

- Let $k_0 = -\infty$ and $k_d = +\infty$. For each k in subtree c_i, $i = 1, \ldots, d$, we have that

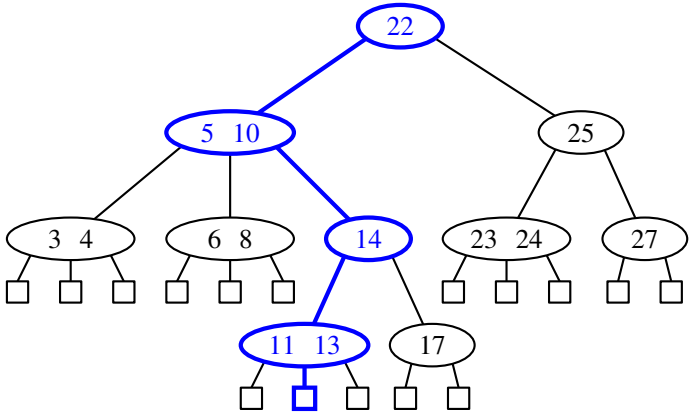
$$k_{i-1} \leq k \leq k_i.$$



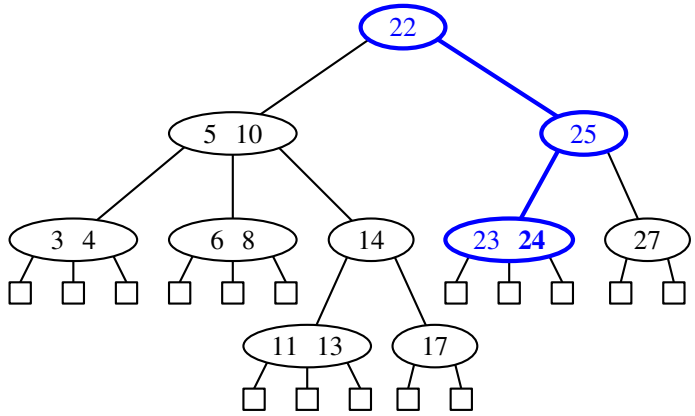
Multiway search tree



a multiway search tree



search path for 12



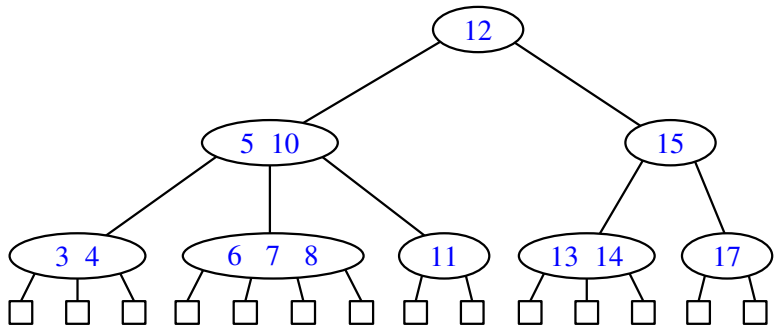
search path for 24

A (2,4) tree

Definition of (2,4) Tree:

Size Property: Every internal node has 2 to 4 four children (d-node with d=2, 3, or 4)

Depth Property: All the external nodes have the same depth.



Proposition

The height of a (2, 4) tree storing n entries is O(log n).

Insertion operation

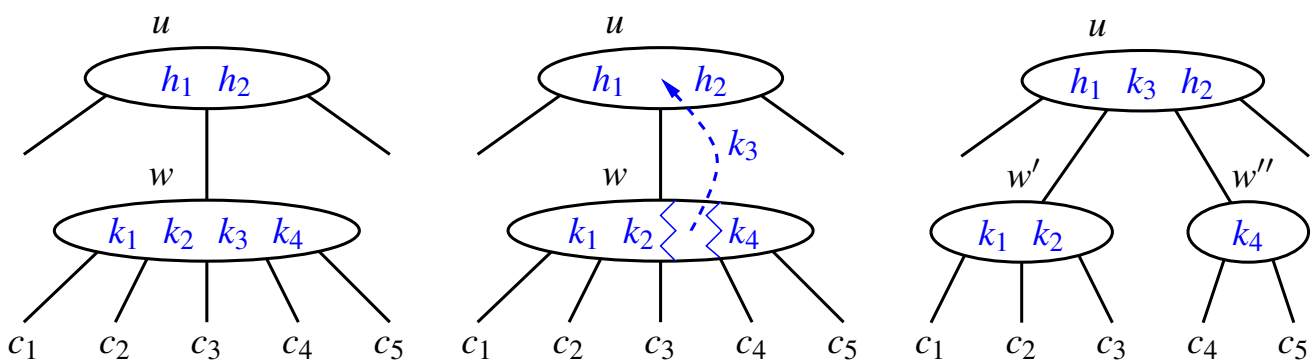
Algorithm 1: put(k,v)

```
1  Search for k
2  if search ends at an external node z then
   |   // T does not contain k
3  |   Let w be the parent of z;
4  |   insert k into node w;
5  |   add a new child y (an external node) to w on the left of z
6  |   if w was previously a 4-node then
7  |   |   split(w)
8  |   end
9  end
10 else
11 |   replace the value of k with v
12 end
```

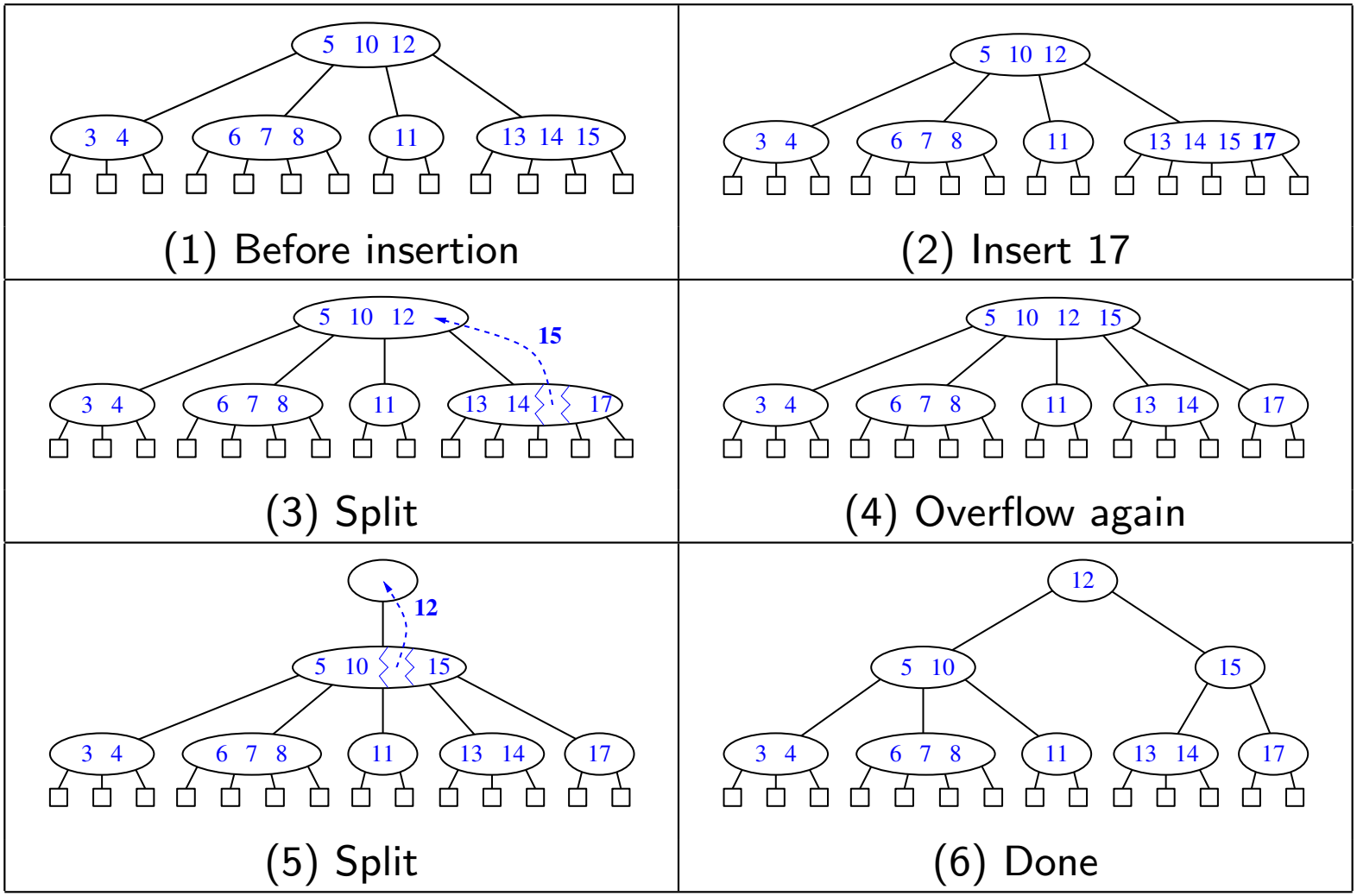
Node split

Algorithm 2: Split(w)

if *w is the root of T* **then**
    create a new root node *u*
else
    let *u* be the parent of *w*.
Replace *w* with two nodes *w‘* and *w“*
w‘ is a 3-node with children c_1, c_2, c_3 storing keys k_1 and k_2.
w“ is a 2-node with children c_4, c_5 storing key k_4.
Insert key k_3 into *u* and make *w‘* and *w“* children of *u*
if *w was child i of u* **then**
    w‘ = child i of u
    w“= child (i + 1) of u



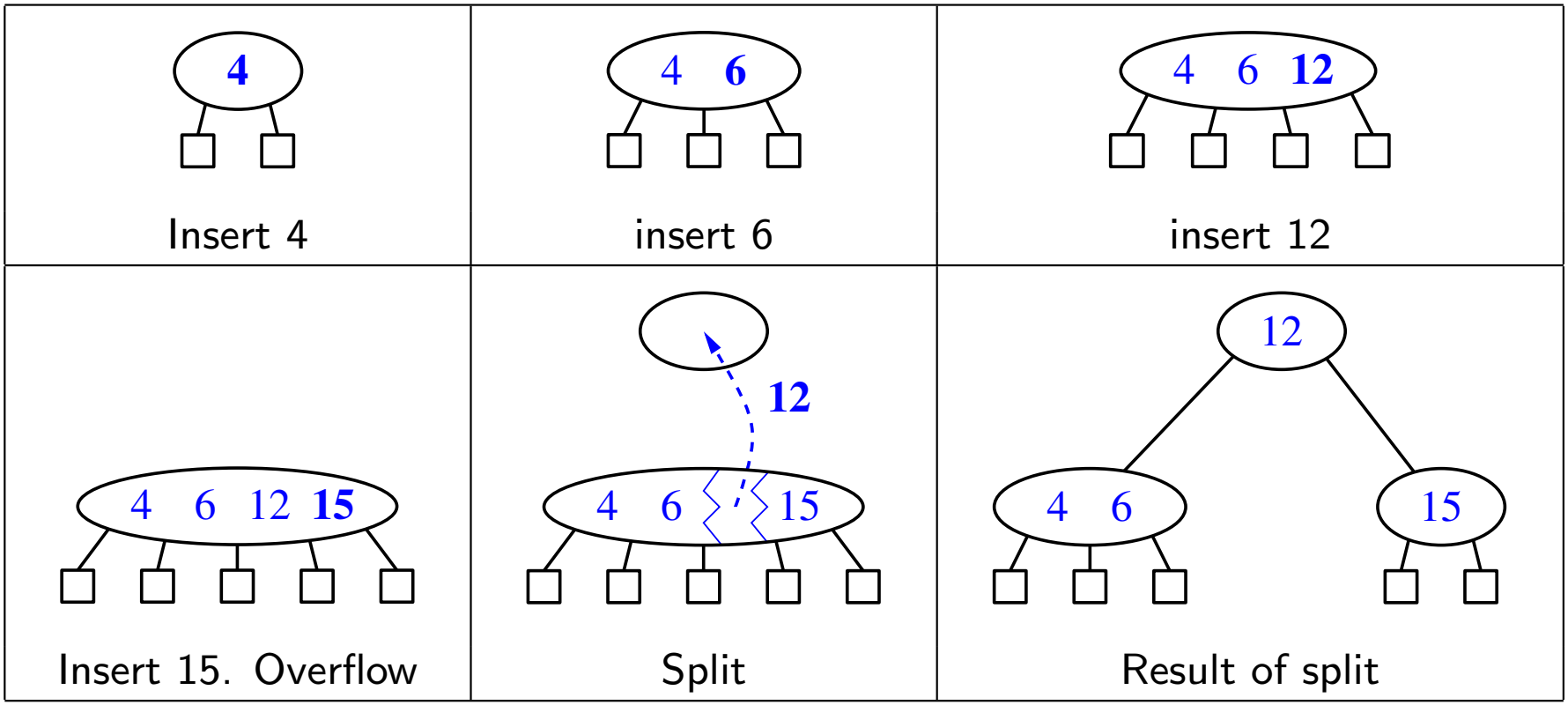
Cascading split



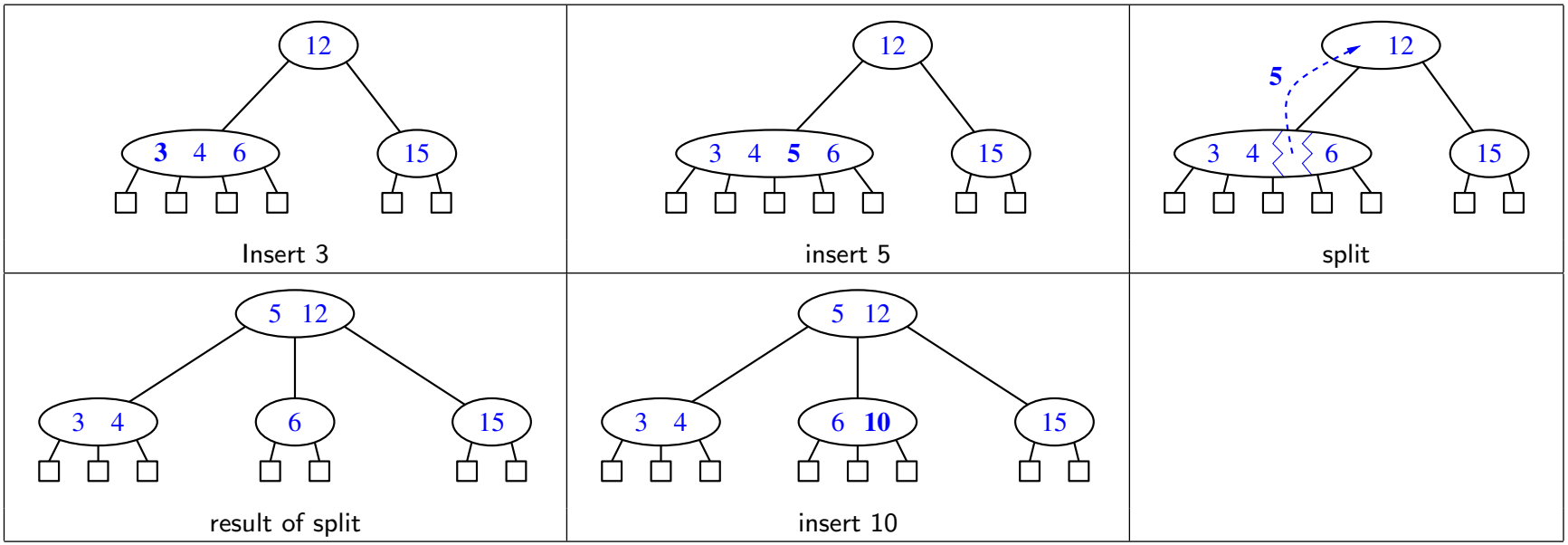
Ex. Del 7, 3, 4, 12, 13, 14

Ex. Insert 4, 6, 12, 15, 3, 5, 10, 8, 11, 7, 13, 14, 17

Insertion Example



Insertion Example



Deletion

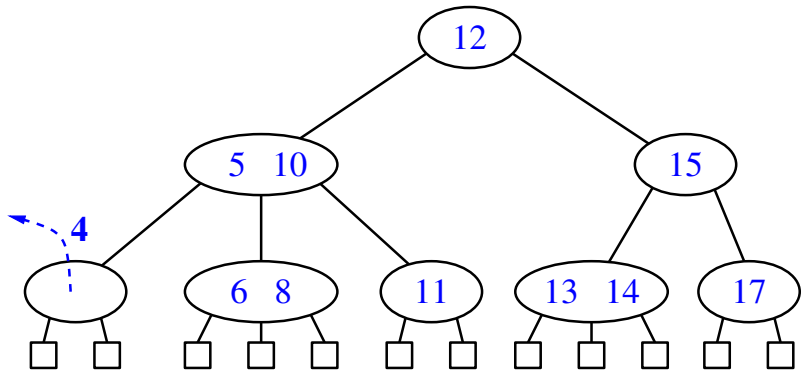
Algorithm 3: remove(k,u)

```
1 search for k. Suppose we find it in node w.
2 if w is not at bottom then
3     swap it with the rightmost bottom node rooted from the left child of w.
4 remove entry k from w;
5 if w underflows (having 0 entry) then
6     if immediate sibling is a 3-node or 4-node then
7         do a transfer op
8     else
9         do a fusion operation (merge with a sibling, move an entry down)
```

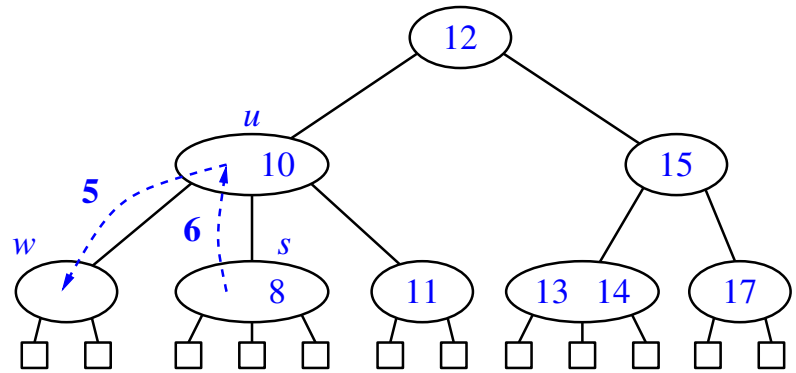
Algorithm 4: tansfer(w)

```
1 move an entry in sibling to parent;
2 move an entry in parent to w
```

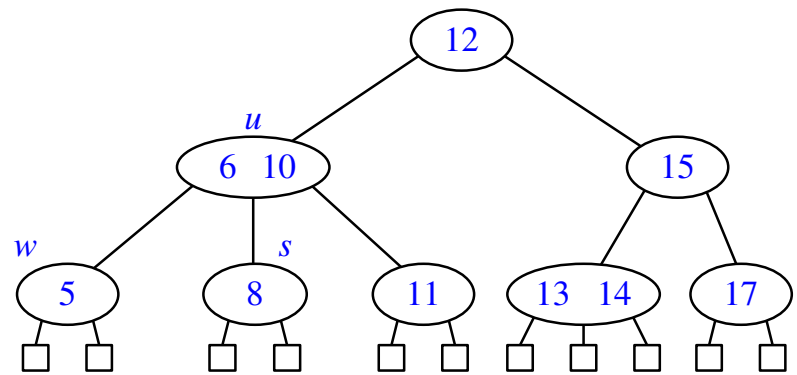
Deletion sequence



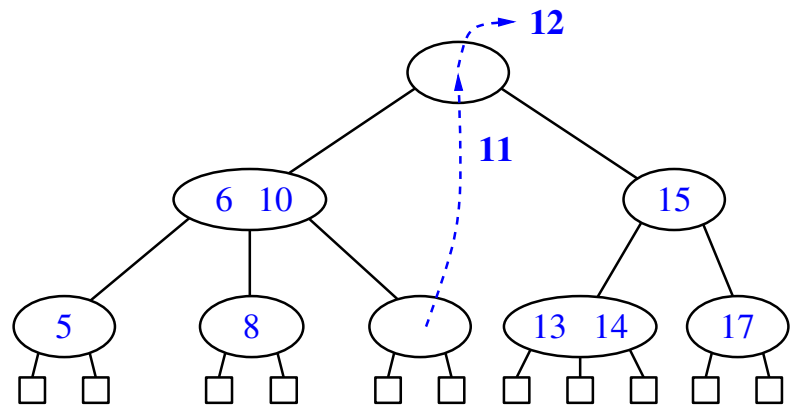
Delete 4



transfer from a sibling

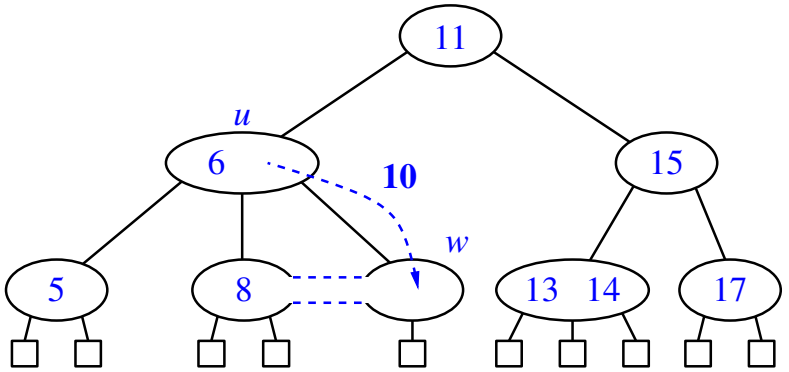


result of transfer op

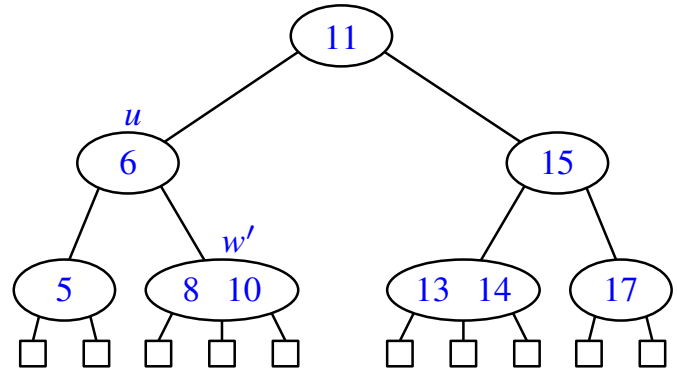


Delete 12, swap

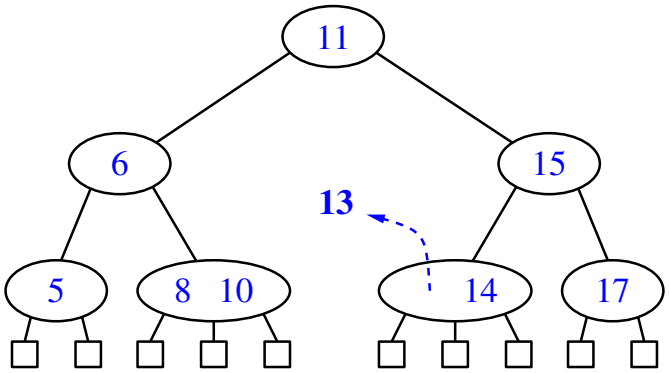
Deletion



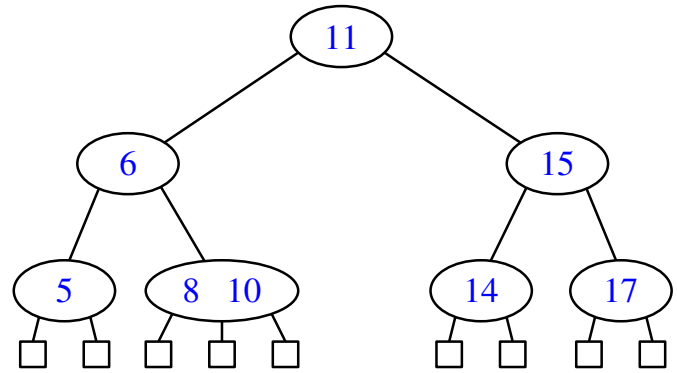
Fusion op.



Result of fusion

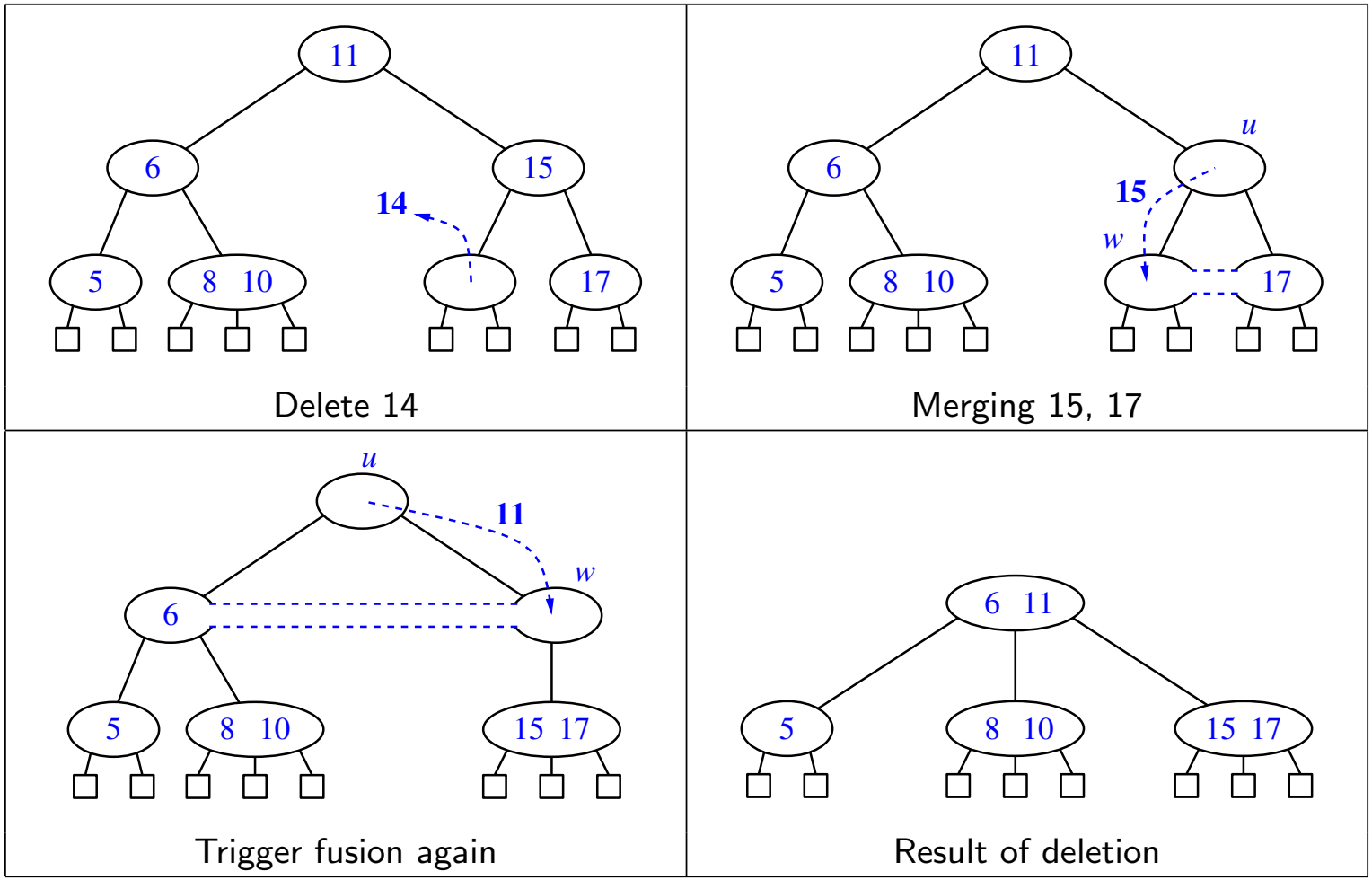


Delete 13



Result of deletion

Propagation of fusion



Binary Search Tree

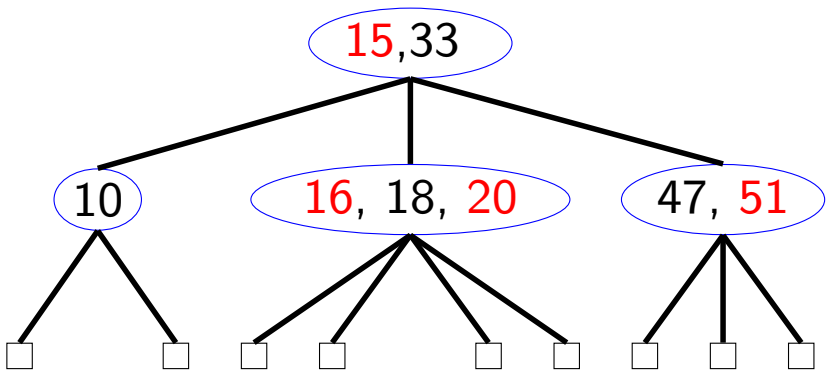
Balanced Search Tree

AVL Tree

(2,4) trees

Red-black trees

Can we transform a (2,4) tree into a 'normal' BST?



Red-black tree

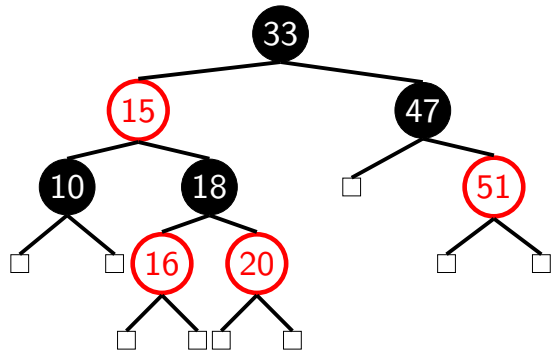
A red-black tree is a binary search tree with properties:

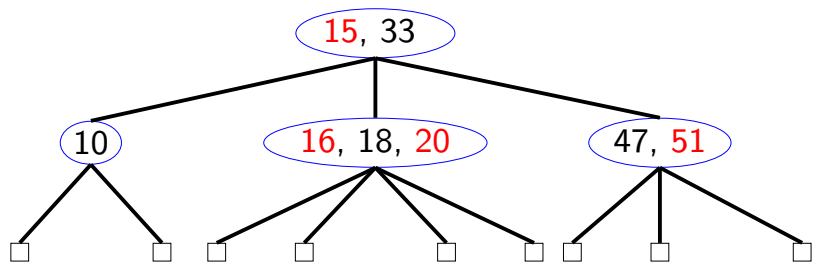
Root property The root is black

External property Every leaf (NULL) is black.

Red property If a node is red, then both its children are black. No double red.

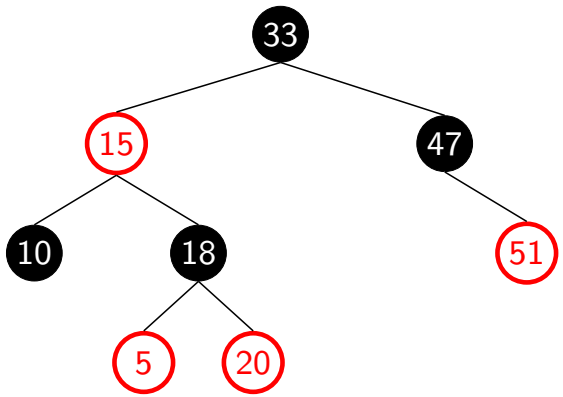
Black depth property Every path from a node to a descendant leaf contains the same number of black nodes.



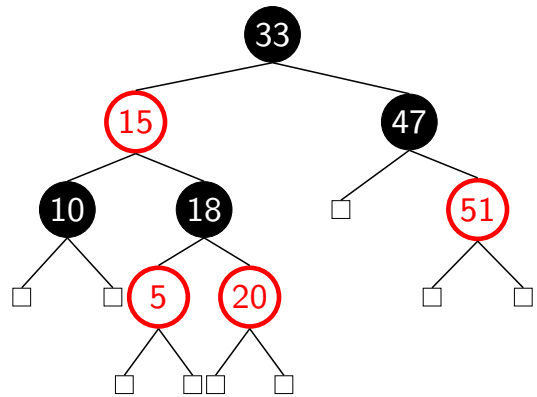


Trees with/without sentinel nodes

- Search tree algorithms usually include the sentinel nodes as a convenient means of flagging that you have reached a leaf node.
- Black height of a node: The number of black nodes on any path from the node.



Basic red-black tree



Sentinel nodes added

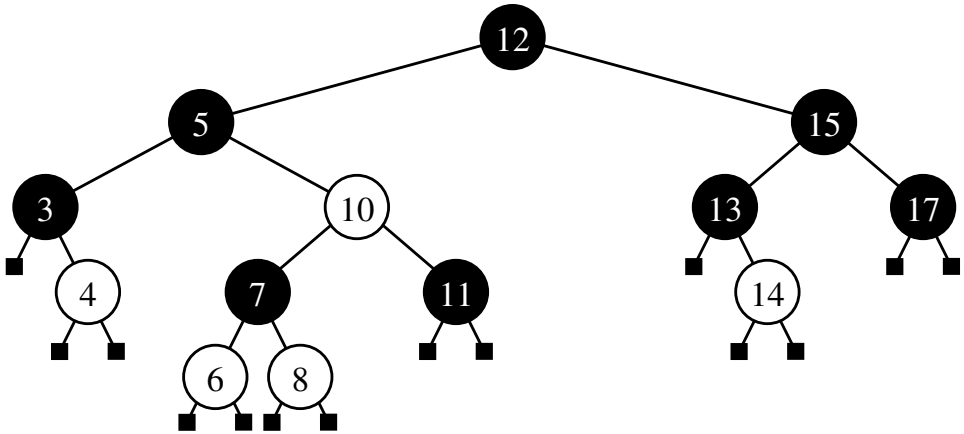
Property: Black Height of the tree

A red-black tree with n internal nodes has height at most $2\log(n+1)$.

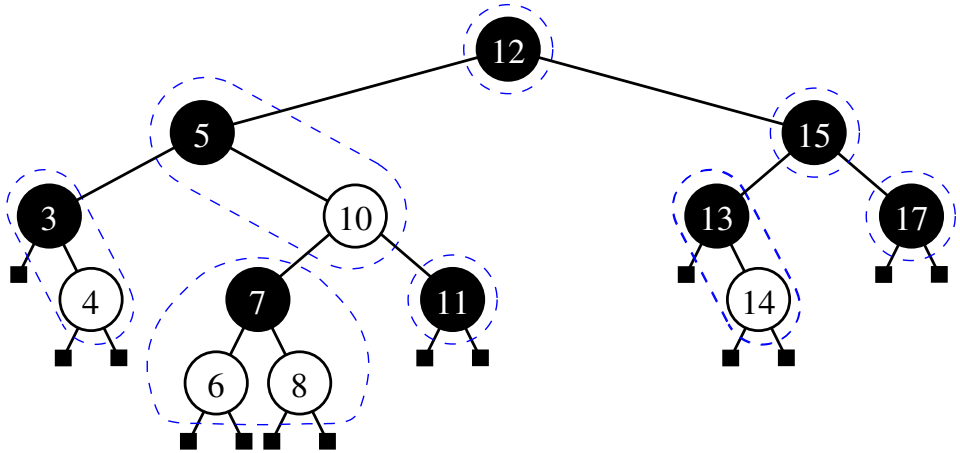
For a proof, see Goodrich et al., Page 512

- This demonstrates why the red-black tree is a good search tree: it can always be searched in $O(\log n)$ time.
- As with AVL and (2,4) trees and heaps, additions to and deletions from red-black trees destroy the red-black property. So we need to restore it.
- To do this we need to look at some operations on red-black trees.

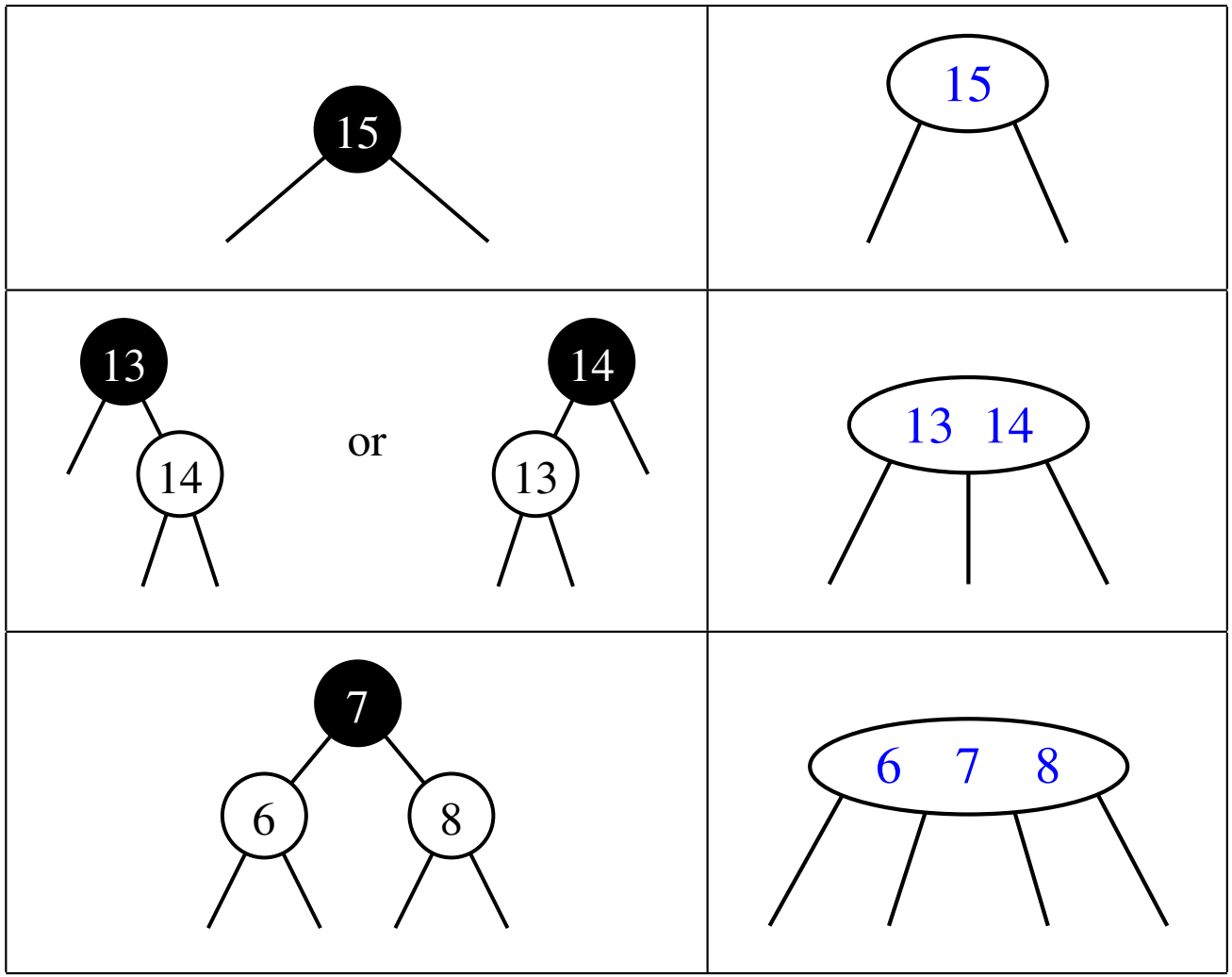
Red-black tree vs. 2-4 tree



RB tree can be transformed into a 2-4 tree, and vice-versa:



(2,4) tree vs red-black tree



Insertion

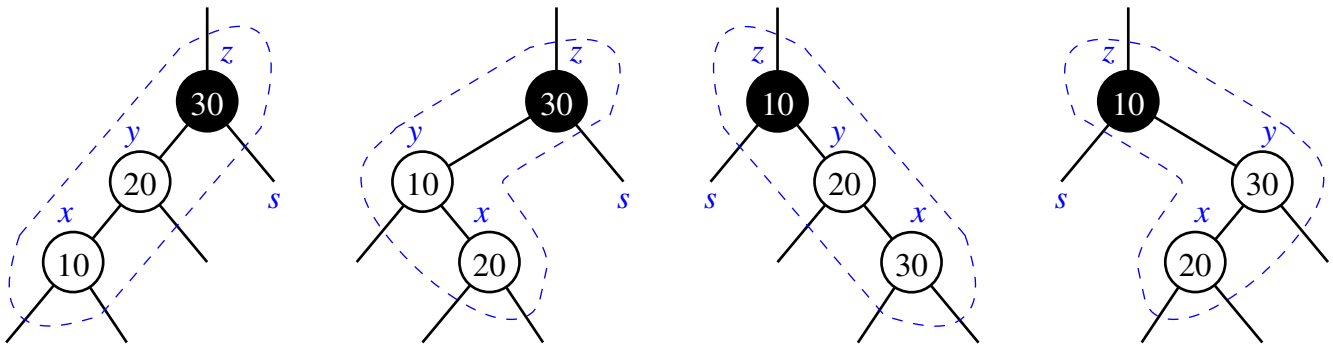
Algorithm 5: put(k,v)

```
1  Search for k, find node x;
2  x.colour=red (if x is not root);
3  y=parent(x);
4  if double red (i.e., x and y are red) then
5      s=uncle(x);
6      if s is black then
7          (a,b,c)=restructure(x),
8          b.color =black
9          a.color =c.color =red.
10     else
11         s.colour=y.colour=black;
12         z.colour=red;
13         if (z is root) z.color=black;
14         continue up to root if double-red occurs again;
```

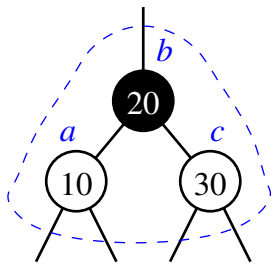
Restructure procedure

Algorithm 6: restructure(x)

1 y=parent(x);
2 z=parent(y);
3 Replace z with the node b (b having medium value);

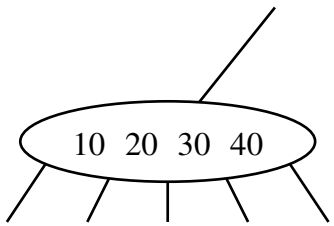


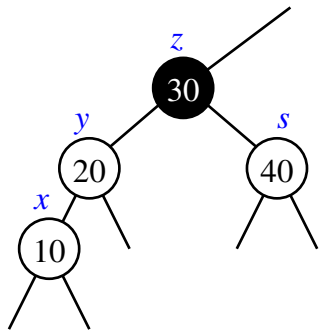
are restructured into

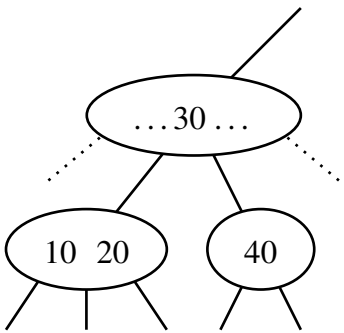


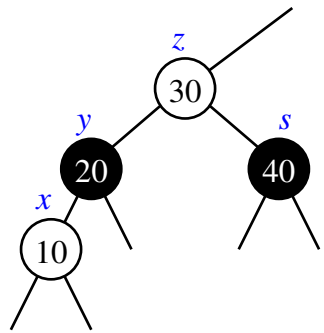
Recolouring procedure

- Recolouring is similar to splitting a 5-node:
  - The uncle is red means that it is a 5-node
  - When we split, one additional layer is added. Hence colour has to change

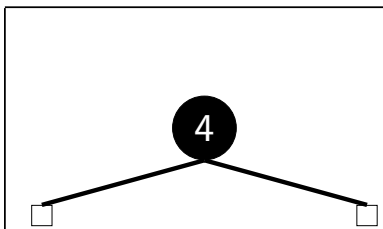




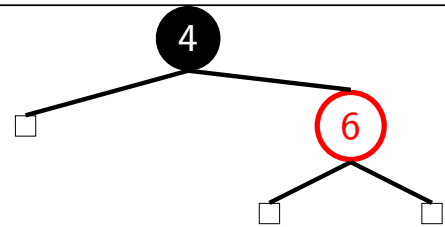




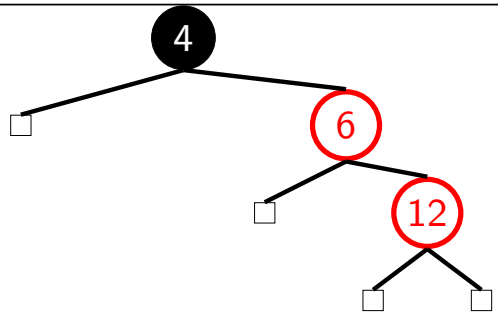
Sequence of insertions



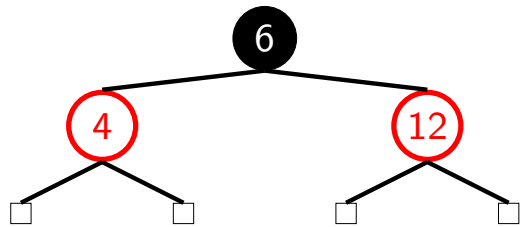
insert 4



insert 6

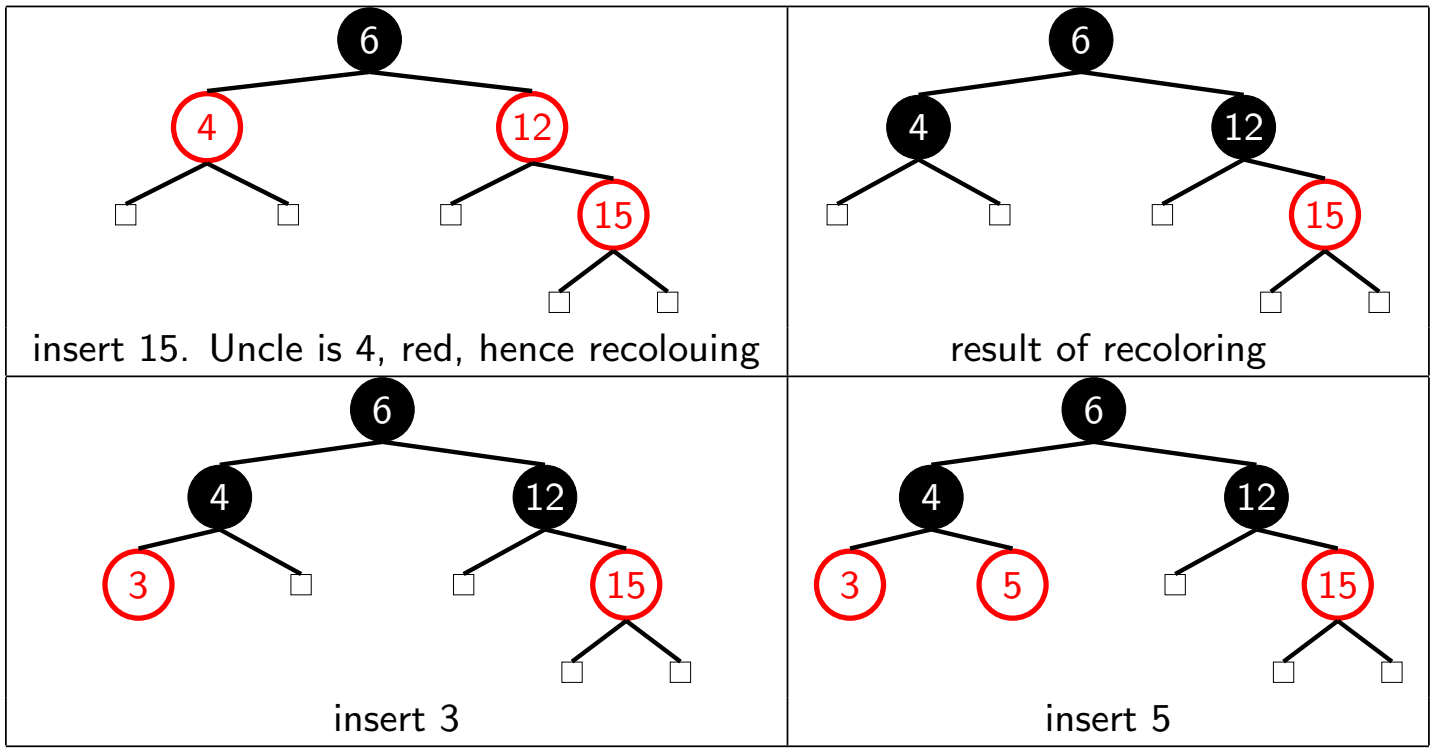


insert 12. uncle is an external node, black. Hence restructure(12)

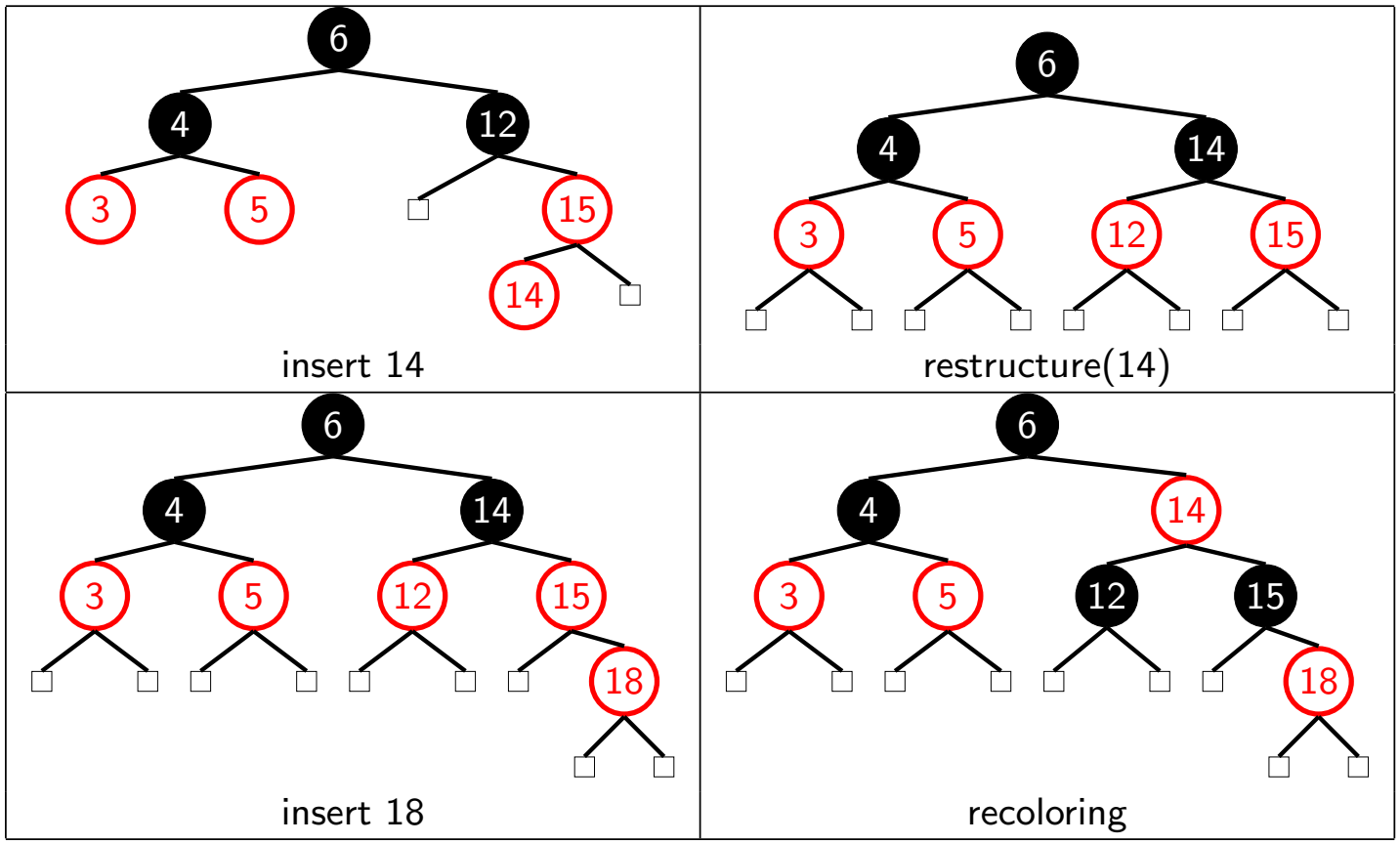


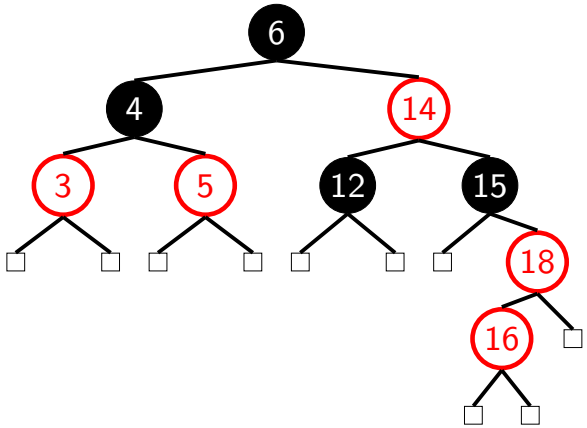
result of restructure

Sequence of insertions

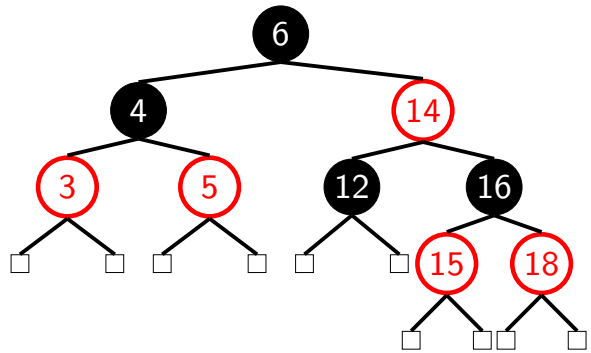


Sequence of insertions

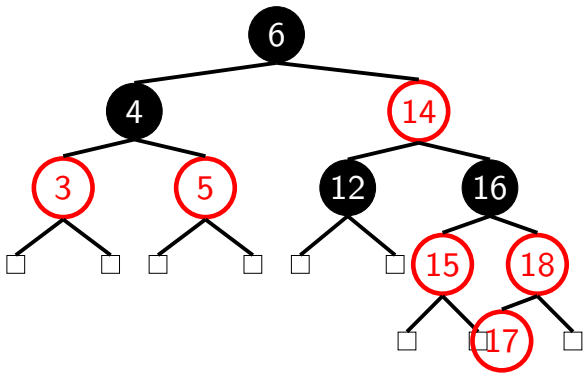




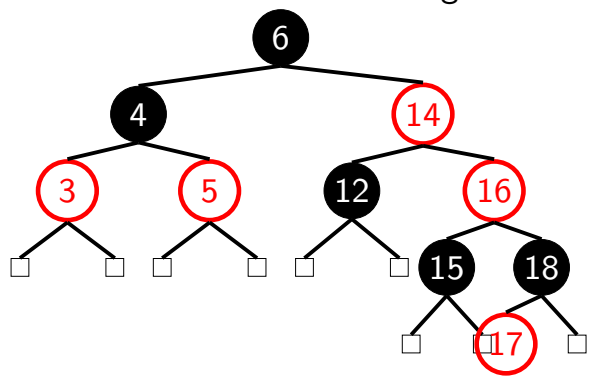
insert 16. double red. uncle is black. restructure(16).



result of restructuring



insert 17. double red. recolouring.

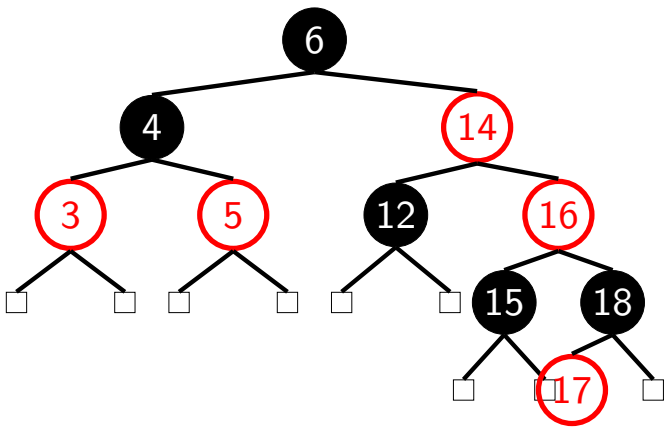


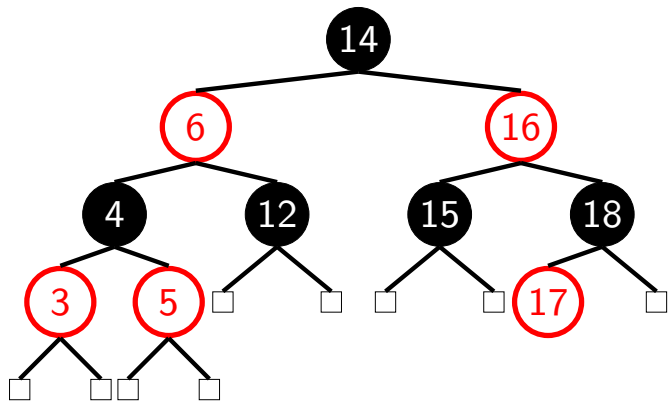
recoloring causing double red again. restructure(16).

Propagating the change

Restructure(16):

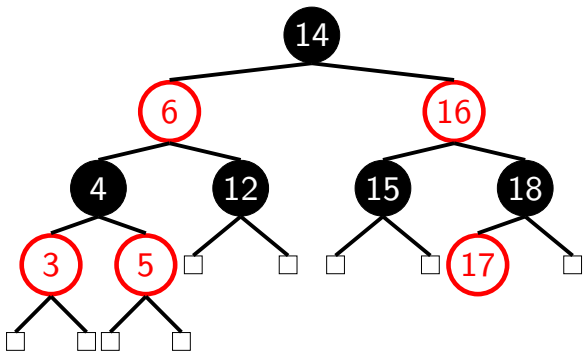
- Raise node 14
- Relink node 12
- Color 14 black, node 6 red.

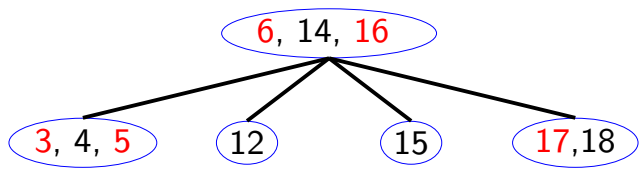




Delete

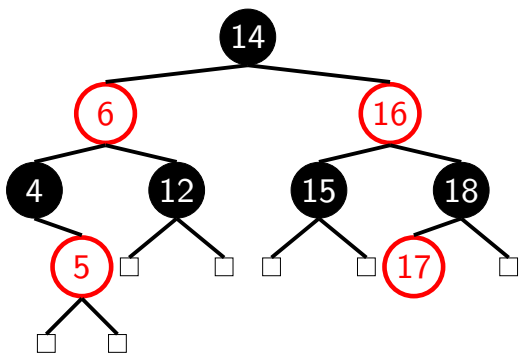
- Delete 3, 12, 17, 18, 16, 15.
- Delete 3:
  - If it is red, delete as usual.

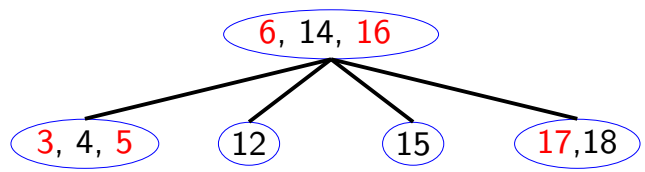




Delete

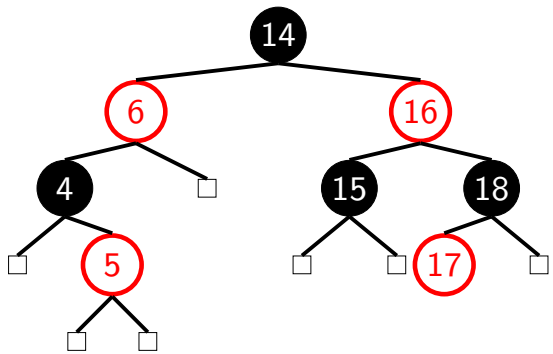
- Delete 3, 12, 17, 18, 16, 15.
- Delete 3:
  - If it is red, delete as usual.

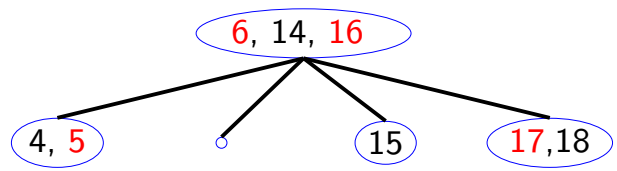




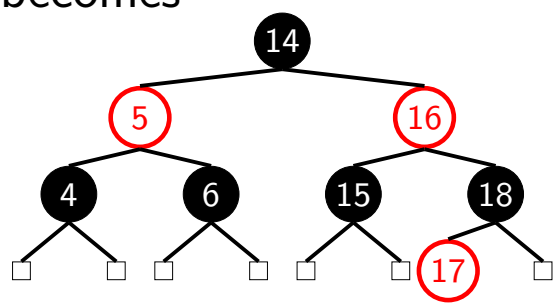
Delete 12 (black node)

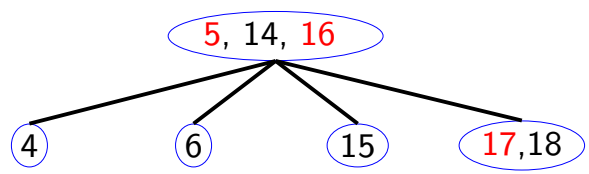
- When a black node is deleted, the 'black height' property is violated.
- Delete 12 (case 1):
  - if sibling is black and has red child x, *restructure*(x).
  - here *restructure*(5)
- Corresponds to the *transfer* operation in (2,4)-tree.





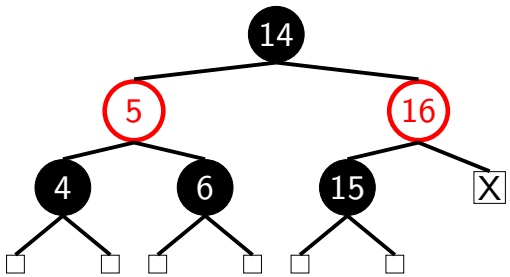
becomes

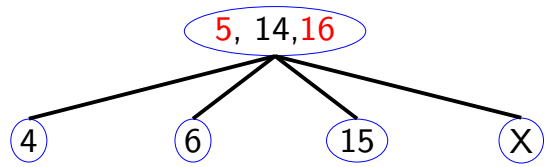




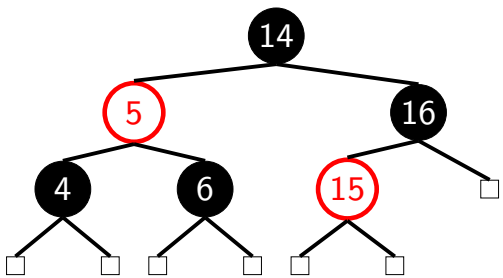
Delete 17, 18

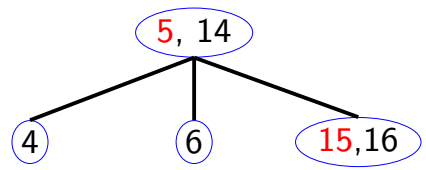
- Delete 18 (case 2):
  - if sibling x is black and both children are black, $recolor(x, parent(x))$.
  - here we $recolor(15, 16)$





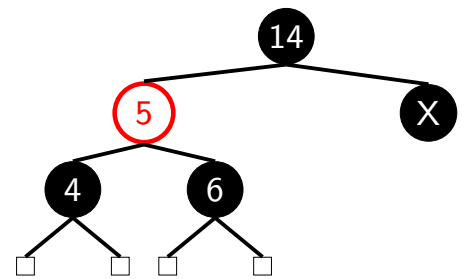
becomes

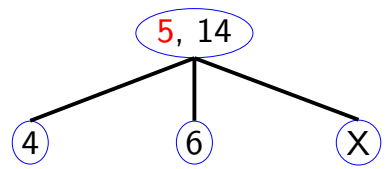




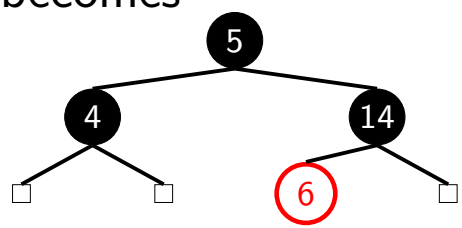
Delete 15, 16

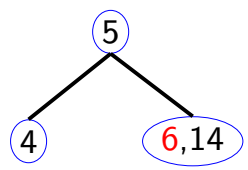
- Delete 16 (case 3):
- if sibling x is red, $rotate(x, parent(x))$. then apply case 1 or case 2.
- here we $rotate(5, 14)$, and $recolor(5, 6)$





becomes





Algorithm for deletion

Algorithm 7: delete

```
1  if it is red then
2  |   delete as usual;
3  end
4  else
5  |   // deleted node is black
6  |   if the sibling is black then
7  |   |   if the sibling has a red child then
8  |   |   |   Do a transfer (restructure (sibling.child))
9  |   |   end
10 |   |   else
11 |   |   |   Do a fusion (recolor (sibling, parent))
12 |   |   end
13 |   end
14 |   else
15 |   |   // sibling is red
16 |   |   Do a fusion (Rotate and re-color)
17 |   end
18 end
```

Takeaways

- BST, why BST. Search, insertion, and remove operations. How are they implemented. What are the complexities for those operations?
- AVL tree. Definition. why AVL trees? (balanced search tree). What is the height of AVL? How to implement those three operations? What are the complexities for those three operations in AVL trees?
- (2,4)-tree. Definition. What is the height. The algorithm and complexities for those three operations.
- Red-black tree. Definition. The algorithm and complexities for those three operations.
- Readings: Goodrich et al. P460-487, P500-524.